Contents

JavaScript Function লেখার প্রকারভেদ 9

১. Function Declaration (নামযুক্ত ফাংশন) 9

২. Function Express (Placeholder1)**ion (অ্যানোনিমাস বা নামযুক্ত)** 9

৩. Arrow Function (ES6) 10

৪. IIFE (Immediately Invoked Function Expression) 10

৫. Named Function Expression 10

৬. Constructor Function 11

৭. Generator Function 11

৮. Async Function 11

৯. Method (Object-এর ভেতরে ফাংশন**)** 12

১০. Callback Function 12

১১. Higher-Order Function 12

১২. Default Parameter Function 13

১৩. Rest Parameter Function 13

১৪. Spread-এর সাথে Call 13

JavaScript Event লেখার প্রকারভেদ 13

১. Inline HTML Event Handler (পুরনো পদ্ধতি**)** 14

২. DOM Property Event Handler 14

৩. addEventListener() —Recommanded 14

৪. Named Function as Handler 15

৫. Arrow Function as Handler 15

৬. Event Delegation 15

৭. Capturing Phase Handler 16

৮. Once Event Handler 16

৯. Passive Event Handler 16

১০. Custom Event 17

১১. Inline Event via Property (Arrow style) 17

১২. Event Listener with AbortController 17

JavaScript Array লেখার প্রকারভেদ 18

Array লেখার ১০টি উপায় 18

১. Array Literal (সবচেয়ে common) 18

২. Array Constructor 19

৩. Array.of() 19

৪. Array.from() 19

৫. Spread Operator (...) 20

৬. Destructuring (Array থেকে মান বের করা) 20

৭. Multidimensional / Nested Array 20

৮. Empty Array then Push 20

৯. Typed Arrays 21

১০. Array from Set / Map 21

JavaScript Loop লেখার প্রকারভেদ 21

Loop লেখার ২০টি উপায় 21

১. for Loop (Classic) 22

২. while Loop 22

৩. do...while Loop 23

৪. for...of Loop (ES6) 23

৫. for...in Loop 23

৬. forEach() Method 24

৭. map() Method (Loop-এর মতো) 24

৮. filter() Method 24

৯. reduce() Method 25

১০. some() Method 25

১১. every() Method 25

১২. find() Method 25
১৩. findIndex() Method 26
১৪. break Statement 26
১৫. continue Statement 26
১৬. Nested Loop 27
১৭. Labeled Loop 27
১৮. Recursion (Loop-এর বিকল্প) 28
১৯. for await...of (Async Loop) 28
২০. for...of with entries() 28
JavaScript Condition লেখার প্রকারভেদ 28
Condition লেখার ২০টি উপায় 29
১. if Statement 29
২. if...else Statement 30
৩. if...else if...else Statement 30
৪. Nested if 31
৫. Ternary Operator (?:) 31
৬. Nested Ternary 31
৭. switch Statement 32
৮. switch with Fall-through 32
৯. Logical AND (&&) Short-circuit 33
১০. Logical OR (||) Short-circuit 33
১১. Nullish Coalescing (??) 33
১২. Optional Chaining (?.) 33
১৩. Logical Assignment Operators 34
১৪. Object Lookup (Switch-এর বিকল্প) 34
১৫. Array.includes() as Condition 34
১৬. Truthy / Falsy Check 35

১৭. try...catch as Condition (Error handling) 35

১৮. Guard Clause (Early Return) 36

১৯. Conditional Function Call 36

২০. Pattern Matching (switch true) 37

JavaScript This লেখার প্রকারভেদ 37

`this` কী? 37

`this` কীভাবে determine হয় — ৬টি Rule 38

Rule ১: Global Context 38

Rule ২: Object Method (Implicit Binding) 39

Rule ৩: Explicit Binding (call, apply, bind) 40

Rule ৪: Constructor Function (new Binding) 40

Rule ৫: Arrow Function (Lexical this) 41

Rule ৬: Event Handler-এ this 42

this না দিলে কী হবে? বিকল্প কী? 42

বিকল্প ১: Hardcoded Object Name 42

বিকল্প ২: Closure Variable 43

বিকল্প ৩: Function Parameter 43

বিকল্প ৪: Class / Modern Approach 43

this না দিলে কী কী সমস্যা হয়? 44

সমস্যা ১: Object-এর নিজের data access করা যায় না 44

সমস্যা ২: Method reuse করা যায় না 44

সমস্যা ৩: Callback-এ reference হারায় 44

সমস্যা ৪: Subclass-এ parent method কাজ করে না 44

সমস্যা ৫: Dynamic property access করা যায় না 44

`this` বোঝার Master Table 45

`this` বের করার সহজ নিয়ম (Priority Order) 45

Frontend Developer হিসেবে Must Know 45

সারসংক্ষেপ 46

JavaScript Return লেখার প্রকারভেদ 46

`return` কী? 47

কেন `return` দরকার? 47

`return` কীভাবে কাজ করে — ৬টি Rule 47

Rule ১: Value Return করা 47

Rule ২: `return` না দিলে `undefined` Return হয় 48

Rule ৩: Early Return (Guard Clause) 48

Rule ৪: Multiple Value Return (Object/Array) 49

Rule ৫: Function Return করা (Higher-Order) 49

Rule ৬: `return` Execution থামায় 50

`return` vs `console.log` — **বড় পার্থক্য** 50

`return` না দিলে কী হবে? বিকল্প কী? 51

বিকল্প ১: Global Variable-এ Store করা 51

বিকল্প ২: Callback-এ পাঠানো 51

বিকল্প ৩: Object-এ Store করা 52

তুলনা: `return` কেন সেরা? 52

`return` না দিলে কী কী সমস্যা হয়? 52

সমস্যা ১: Value ব্যবহার করা যায় না 52

সমস্যা ২: Chain করা যায় না 53

সমস্যা ৩: Array Method-এ ভুল ফলাফল 53

সমস্যা ৪: Function Composition ভাঙে 53

সমস্যা ৫: Async Function-এ Value হারায় 54

সমস্যা ৬: Condition-এ ভুল Branch 54

সমস্যা ৭: Recursion ভাঙে 54

Implicit vs Explicit Return 55

Explicit Return 55

Implicit Return (শুধু Arrow Function) 55

`return`-এর বিশেষ ব্যবহার 55
১. Void Function (return কিছু নেই) 55
২. `return` এরপর newline সাবধান 55
৩. Constructor-এ `return` 56
৪. Async Function-এ `return` 56
৫. Generator-এ `return` 56
`return` Master Table 56
বাস্তব উদাহরণ — সব Rule একসাথে 57
Must Know 57
সারসংক্ষেপ 58
JavaScript Number Buildin object & Function 58
১. `Number` Constractor ও Static method 59
২. `Math` নেমস্পেস অবজেক্ট 59
৩. `Number.prototype` মেথড (প্রিমিটিভ নাম্বার ইনস্ট্যান্স) 60
৪. `BigInt` (আধুনিক সংযোজন) 60
সারসংক্ষেপ (কাজের ভিত্তিতে গ্রুপিং) 60
JavaScript String Buildin object & Function 60
১. `String` কনস্ট্রাক্টর ও স্ট্যাটিক মেথড 60
২. `String.prototype` এর প্রপার্টি (Instance Properties) 60
৩. `String.prototype` এর মেথড (Instance Methods) 61
৪. `String.prototype` এর HTML র‍্যাপার মেথড (HTML Wrapper Methods) 62
JavaScript Symbol Buildin object & Function 62
১. Symbol কনস্ট্রাক্টর ও গ্লোবাল ফাংশন 62
২. সুপরিচিত Symbol (Well-Known Symbols) — স্ট্যাটিক প্রপার্টি 62
৩. Symbol প্রোটোটাইপের প্রপার্টি ও মেথড (Instance Properties & Methods) 63
৪. Symbol-সংক্রান্ত ইউটিলিটি ও অবজেক্ট আচরণ 63
সারসংক্ষেপ 64
JavaScript Array Buildin object & Function 64

১. স্ট্যাটিক মেথড (Static Methods) 64
২. মিউটেটিং মেথড (Mutating Methods) 64
৩. অ্যাক্সেসর বা নন-মিউটেটিং মেথড (Accessor / Non-Mutating Methods) 65
৪. ইটারেশন বা কলব্যাক মেথড (Iteration Methods) 65
প্রপার্টি (Properties) 66
JavaScript Date Buildin object & Function 66
১. স্ট্যাটিক মেথড (Static Methods) 66
২. গেটার মেথড (Getter Methods / "get" Methods) 67
৩. সেটার মেথড (Setter Methods / "set" Methods) 67
৪. ফরম্যাটিং বা "to" মেথড (Formatting / "to" Methods) 67
৫. কনস্ট্রাক্টর (Constructor) 68
JavaScript DOM Manipulation 68
1. Element Selection 68
6. Class Manipulation (ক্লাস) 70
8. Traversal 70
9. Events 70
10. Form Handling 71
11. Dimension & Position 71
12. Document Level 72
13. Window Object 72
14. Storage (স্টোরেজ) 73
15. Miscellaneous 73
16. HTMLCollection vs NodeList 73
17. Common Properties 73
18. Timing & Performance 74
19. Best Practices 74
20. Quick Cheatsheet 74
JavaScript BOM Manipulation 75
BOM হলো Browser Object Model এটি JavaScript এর মাধ্যমে browser এর সাথে interact করার interface। BOM এর কোনো official standard নেই 75
2. Window Size & Position 75

3. Window Methods (Opening & Closing) 75
4. Window Scroll Methods 76
5. Dialog Boxes (Dialogs) 76
6. Timers 76
8. History Object 77
9. Navigator Object 77
10. Screen Object 78
11. Frames & Iframe 78
12. Storage (Storage APIs) 79
LocalStorage 79
SessionStorage 79
IndexedDB 79
Cookies 79
13. Console Object 80
15. Geolocation API 80
16. Web Storage & Cache APIs 81
18. Notification API 81
19. Clipboard API 82
20. Media & Device APIs 82
21. Permissions API 82
22. Performance API 82
23. Crypto API 82
24. Encoding & URL APIs 83
25. Frame & Window Communication 83
26. Other Window Utilities 83
29. Best Practices 84
30. Browser Support Notes 84

JavaScript Function লেখার প্রকারভেদ

Minimum জানতে হবে (Must Know):

1. Function Declaration (বেসিক, সব জায়গায়)
2. Arrow Function (Modern JS, React, callbacks)
3. Callback Function (Event handling, async)
4. Async/Await Function (API calls, data fetching)
5. Function Expression (Variables, module patterns)

Good to Know:

6. Method (Object-এর ভেতরে ফাংশন , Object-oriented code)
7. Default Parameter (Clean API design)
8. Rest Parameter (Flexible functions)
9. IIFE (Scope isolation)
10. Higher-Order Function (React, functional patterns)

Advanced:

11. Generator Function (Advanced async, iterators)
12. Constructor Function (Legacy OOP)
13. Named Function Expression (Recursion, debugging)
14. Spread with Call (Utility functions)

Tip: শুরুতে ৫টি Must Know ফাংশন ভালোভাবে আয়ত্ত করুন, বাকিগুলো প্রয়োজন অনুযায়ী শিখুন।

১. Function Declaration (নামযুক্ত ফাংশন)

কখন কাজে লাগে: সাধারণ ফাংশন, হোস্টিং সুবিধা দরকার হলে, বারবার ব্যবহারের জন্য।

Syntax:

```
function functionName(parameters) {
  // code
  return value;
}
```

Example:

```
function add(a, b) {
  return a + b;
}
console.log(add(5, 3)); // 8
```

---

২. Function Express (Placeholder1)ion (অ্যানোনিমাস বা নামযুক্ত)

কখন কাজে লাগে: ভেরিয়েবলে ফাংশন স্টোর করা, callback হিসেবে পাঠানো, হোস্টিং দরকার না হলে।

Syntax:

```
const functionName = function(parameters) {
  // code
};
```

Example:

```
const multiply = function(a, b) {
  return a * b;
};
```

```
console.log(multiply(4, 5)); // 20
```

---

৩. Arrow Function (ES6)

কখন কাজে লাগে: শর্ট সিনট্যাক্স, `this` bind করতে না চাইলে (lexical this), array methods-এ।

Syntax:

```
const functionName = (parameters) => {
  // code
};
// Single line
const functionName = (parameters) => expression;
```

Example:

```
const subtract = (a, b) => a - b;
console.log(subtract(10, 4)); // 6

const numbers = [1, 2, 3];
const doubled = numbers.map(n => n * 2);
console.log(doubled); // [2, 4, 6]
```

---

৪. IIFE (Immediately Invoked Function Expression)

কখন কাজে লাগে: স্কোপ তৈরি করতে, গ্লোবাল namespace দূষিত না করতে, একবারই চালানোর জন্য।

Syntax:

```
(function() {
  // code
})();
// Arrow version
(() => {
  // code
})();
```

Example:

```
(function() {
  const message = "Hello from IIFE";
  console.log(message);
})();
```

---

৫. Named Function Expression

কখন কাজে লাগে: Recursion-এ, stack trace-এ নাম দেখতে।

Syntax:

```
const variableName = function functionName(parameters) {
  // code
};
```

Example:

```
const factorial = function fact(n) {
  return n <= 1 ? 1 : n * fact(n - 1);
};
console.log(factorial(5)); // 120
```

---

৬. Constructor Function

কখন কাজে লাগে: `new` keyword দিয়ে object তৈরি করতে।

Syntax:

```
function FunctionName(parameters) {
  this.property = value;
}
```

Example:

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}
const user = new Person("Rahim", 25);
console.log(user.name); // Rahim
```

---

৭. Generator Function

কখন কাজে লাগে: Iterable sequence তৈরি করতে, lazy evaluation-এ, async flow-তে।

Syntax:

```
function* functionName(parameters) {
  yield value;
}
```

Example:

```
function* idGenerator() {
  let id = 1;
  while (true) {
    yield id++;
  }
}
const gen = idGenerator();
console.log(gen.next().value); // 1
console.log(gen.next().value); // 2
```

---

৮. Async Function

কখন কাজে লাগে: Promise handle করতে, asynchronous code synchronous-এর মতো লিখতে।

Syntax:

```
async function functionName(parameters) {
  const result = await promise;
  return result;
}

// Arrow version
const functionName = async (parameters) => {
  const result = await promise;
  return result;
};
```

Example:

```
async function fetchData() {
  const response = await fetch("https://api.example.com/data");
  const data = await response.json();
```

```
  return data;
}
```

---

৯. Method (Object-এর ভেতরে ফাংশন)

কখন কাজে লাগে: Object-এর behavior define করতে।

Syntax:

```
const object = {
  methodName(parameters) {
    // code
  }
};
```

Example:

```
const calculator = {
  add(a, b) {
    return a + b;
  }
};
console.log(calculator.add(3, 7)); // 10
```

---

১০. Callback Function

কখন কাজে লাগে: অন্য ফাংশনের argument হিসেবে পাঠাতে, event handling-এ।

Syntax:

```
function mainFunction(callback) {
  callback();
}
```

Example:

```
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}
greet("Karim", () => console.log("Welcome!"));
```

---

১১. Higher-Order Function

কখন কাজে লাগে: ফাংশন return করতে বা ফাংশন argument নিতে (functional programming)।

Syntax:

```
function higherOrder(callback) {
  return function() {
    callback();
  };
}
```

Example:

```
function multiplier(factor) {
  return (number) => number * factor;
}
const double = multiplier(2);
console.log(double(5)); // 10
```

---

১২. Default Parameter Function

কখন কাজে লাগে: Argument না দিলে default value ব্যবহার করতে।

Syntax:

```
function functionName(param = defaultValue) {
  // code
}
```

Example:

```
function greet(name = "Guest") {
  return "Hello " + name;
}
console.log(greet()); // Hello Guest
console.log(greet("Rahim")); // Hello Rahim
```

---

১৩. Rest Parameter Function

কখন কাজে লাগে: অসংখ্য argument একটি array-তে নিতে।

Syntax:

```
function functionName(...args) {
  // code
}
```

Example:

```
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3, 4)); // 10
```

---

১৪. Spread-এর সাথে Call

কখন কাজে লাগে: Array-কে individual argument-এ রূপান্তর করতে।

Syntax:

```
functionName(...array);
```

Example:

```
const nums = [5, 10, 15];
console.log(Math.max(...nums)); // 15
```

JavaScript Event লেখার প্রকারভেদ

1. Inline HTML — `onclick="..."`
2. DOM Property — `el.onclick = fn`
3. addEventListener() — `el.addEventListener(...)` (সব modern code)
4. Named Function Handler — `addEventListener("click", namedFn)`
5. Arrow Function Handler — `addEventListener("click", () => {})`
6. Event Delegation — `parent.addEventListener + e.target.matches`
7. Capturing — `addEventListener(..., true)`
8. Once — `addEventListener(..., { once: true })`

9. Passive — `addEventListener(..., { passive: true })`
10. Custom Event — `new CustomEvent() + dispatchEvent()`
11. Arrow Property Handler — `el.onclick = (e) => {}`
12. AbortController — `{ signal: controller.signal } + controller.abort()`

Must Know ৫টি:

1. `addEventListener()` + Event Object
2. `click`, `submit`, `input`
3. Event Delegation
4. `preventDefault()` / `stopPropagation()`
5. Bubbling vs Capturing

Tip: DOM event concept ভালোভাবে জানলে React, Vue, Svelte — সব framework-এ event handling সহজ হয়ে যায়।

১. Inline HTML Event Handler (পুরনো পদ্ধতি)

কখন কাজে লাগে: ছোট demo/testing-এ। Production-এ ব্যবহার করা উচিত নয় (maintainability খারাপ)।

Syntax:

```html
<element eventname="javascriptCode">
```

Example:

```html
<button onclick="alert('Clicked!')">Click Me</button>
```

---

২. DOM Property Event Handler

কখন কাজে লাগে: একটি মাত্র handler সেট করতে। একই event-এ দ্বিতীয়বার সেট করলে আগেরটা overwrite হয়।

Syntax:

```
element.eventname = function() {
  // code
};
```

Example:

```
const btn = document.querySelector("#myBtn");
btn.onclick = function() {
  console.log("Clicked!");
};
```

---

৩. addEventListener() —Recommanded☑

কখন কাজে লাগে: Production-এ সবসময়। একাধিক handler, capture control, once option সব আছে।

Syntax:

```
element.addEventListener("eventType", handlerFunction, options);
```

Example:

```
const btn = document.querySelector("#myBtn");
btn.addEventListener("click", () => console.log("First"));
btn.addEventListener("click", () => console.log("Second"));
```

---

৪. Named Function as Handler

কখন কাজে লাগে: Handler remove করতে হলে (removeEventListener-এর জন্য named function লাগে)।

Syntax:

```
function handlerName(event) {
  // code
}
element.addEventListener("eventType", handlerName);
```

Example:

```
function handleClick(e) {
  console.log("Clicked:", e.target);
}
btn.addEventListener("click", handleClick);
btn.removeEventListener("click", handleClick);
```

---

৫. Arrow Function as Handler

কখন কাজে লাগে: Short handler, `this` bind করতে না চাইলে।

Syntax:

```
element.addEventListener("eventType", (event) => {
  // code
});
```

Example:

```
btn.addEventListener("click", (e) => {
  console.log("Arrow:", e.type);
});
```

---

৬. Event Delegation

কখন কাজে লাগে: অনেক child element-এ একটি handler দিয়ে কাজ চালাতে (dynamic element-এর জন্য perfect)।

Syntax:

```
parent.addEventListener("eventType", (event) => {
  if (event.target.matches("selector")) {
    // handle
  }
});
```

Example:

```
const list = document.querySelector("ul");
list.addEventListener("click", (e) => {
  if (e.target.tagName === "LI") {
    console.log("Clicked:", e.target.textContent);
  }
});
```

---

৭. Capturing Phase Handler

কখন কাজে লাগে: Event bubble হওয়ার আগে parent-এ ধরতে।

Syntax:

```
element.addEventListener("eventType", handler, true);
// অথবা
element.addEventListener("eventType", handler, { capture: true });
```

Example:

```
parent.addEventListener("click", () => console.log("Capture"), true);
child.addEventListener("click", () => console.log("Target"));
```

---

৮. Once Event Handler

কখন কাজে লাগে: Event শুধু একবার handle করতে।

Syntax:

```
element.addEventListener("eventType", handler, { once: true });
```

Example:

```
btn.addEventListener("click", () => {
  console.log("Only once!");
}, { once: true });
```

---

৯. Passive Event Handler

কখন কাজে লাগে: Scroll/touch performance বাড়াতে (preventDefault block করে না)।

Syntax:
```
element.addEventListener("eventType", handler, { passive: true });
```

Example:
```
window.addEventListener("scroll", () => {
  console.log(window.scrollY);
}, { passive: true });
```

---

১০. Custom Event

কখন কাজে লাগে: নিজের component থেকে custom event dispatch করতে।

Syntax:
```
const event = new CustomEvent("eventName", { detail: data });
element.dispatchEvent(event);
```

Example:
```
const myEvent = new CustomEvent("userLogin", {
  detail: { name: "Rahim", id: 101 }
});

document.addEventListener("userLogin", (e) => {
  console.log("User:", e.detail.name);
});

document.dispatchEvent(myEvent);
```

---

১১. Inline Event via Property (Arrow style)

কখন কাজে লাগে: দ্রুত prototype বানাতে।

Syntax:
```
element.eventname = (event) => {
  // code
};
```

Example:
```
btn.onclick = (e) => console.log("Inline property:", e.type);
```

---

১২. Event Listener with AbortController

কখন কাজে লাগে: একসাথে অনেক listener remove করতে।

Syntax:
```
const controller = new AbortController();
```

```
element.addEventListener("eventType", handler, { signal: controller.signal });
controller.abort();
```

Example:

```
const controller = new AbortController();

btn.addEventListener("click", () => console.log("A"), { signal: controller.signa
l });
btn.addEventListener("click", () => console.log("B"), { signal: controller.signa
l });

controller.abort(); // দুটোই remove
```

JavaScript Array লেখার প্রকারভেদ

Array লেখার ১০টি উপায়

1. Array Literal — `[1, 2, 3]`
2. Array Constructor — `new Array(1, 2, 3)`
3. Array.of() — `Array.of(1, 2, 3)`
4. Array.from() — `Array.from("abc")`
5. Spread Operator — `[...arr]`
6. Destructuring — `const [a, b] = arr`
7. Multidimensional — `[[1,2],[3,4]]`
8. Empty + Push — `const a = []; a.push(x)`
9. Typed Arrays — `new Uint8Array([...])`
10. Set/Map → Array — `[...new Set(arr)]`

Must Know ১০টি Method:

1. `map()` — transform
2. `filter()` — select
3. `reduce()` — aggregate
4. `forEach()` — iterate
5. `find()` / `findIndex()` — search
6. `push()` / `pop()` — add/remove end
7. `slice()` / `splice()` — copy/modify
8. `includes()` / `indexOf()` — check
9. `sort()` / `reverse()` — order
10. Spread `...` / Destructuring — modern syntax

Tip: `map`, `filter`, `reduce` — এই তিনটি functional method সবচেয়ে বেশি কাজে লাগে। React/Vue-এ data render করার সময় প্রায় প্রতিদিনই এগুলো ব্যবহার হবে।

১. Array Literal (সবচেয়ে common) ☑

কখন কাজে লাগে: সাধারণ array তৈরি করতে, সব জায়গায়।

Syntax:
```
const arrayName = [item1, item2, item3];
```

Example:
```
const fruits = ["apple", "banana", "mango"];
console.log(fruits); // ["apple", "banana", "mango"]
```

---

২. Array Constructor

কখন কাজে লাগে: নির্দিষ্ট length-এর খালি array তৈরি করতে।

Syntax:
```
const arrayName = new Array(item1, item2);
const arrayName = new Array(length);
```

Example:
```
const nums = new Array(1, 2, 3);   // [1, 2, 3]
const empty = new Array(5);        // [empty × 5]
```

---

৩. Array.of()

কখন কাজে লাগে: Single number argument দিয়ে array বানাতে (`new Array(5)` সমস্যা এড়াতে)।

Syntax:
```
const arrayName = Array.of(item1, item2, item3);
```

Example:
```
const arr = Array.of(5);   // [5]  (new Array(5) হতো [empty × 5])
const arr2 = Array.of(1, 2, 3); // [1, 2, 3]
```

---

৪. Array.from()

কখন কাজে লাগে: Iterable/array-like (string, NodeList, Set, Map) থেকে array বানাতে।

Syntax:
```
const arrayName = Array.from(iterable, mapFn);
```

Example:
```
const chars = Array.from("hello");     // ["h","e","l","l","o"]
const doubled = Array.from([1,2,3], x => x * 2); // [2,4,6]
```

---

৫. Spread Operator (...)

কখন কাজে লাগে: Array copy, merge, বা iterable থেকে array বানাতে।

Syntax:

```
const arrayName = [...iterable];
const merged = [...arr1, ...arr2];
```

Example:

```
const a = [1, 2];
const b = [3, 4];
const merged = [...a, ...b];  // [1, 2, 3, 4]
const copy = [...a];          // [1, 2]
```

---

৬. Destructuring (Array থেকে মান বের করা)

কখন কাজে লাগে: Array-এর নির্দিষ্ট element variable-এ রাখতে।

Syntax:

```
const [var1, var2, ...rest] = arrayName;
```

Example:

```
const [first, second, ...others] = [10, 20, 30, 40];
console.log(first, second, others); // 10 20 [30, 40]
```

---

৭. Multidimensional / Nested Array

কখন কাজে লাগে: Matrix, grid, table data রাখতে।

Syntax:

```
const arrayName = [[a, b], [c, d]];
```

Example:

```
const matrix = [[1, 2], [3, 4]];
console.log(matrix[1][0]); // 3
```

---

৮. Empty Array then Push

কখন কাজে লাগে: Loop/condition দিয়ে dynamically data জমা করতে।

Syntax:

```
const arrayName = [];
arrayName.push(item);
```

Example:

```
const evens = [];
for (let i = 1; i <= 10; i++) {
  if (i % 2 === 0) evens.push(i);
}
console.log(evens); // [2, 4, 6, 8, 10]
```

---

৯. Typed Arrays

কখন কাজে লাগে: Binary data, performance-critical কাজ (WebGL, audio, file)।

Syntax:

```
const arrayName = new Int8Array(length);
const arrayName = new Float32Array([1.1, 2.2]);
```

Example:

```
const buffer = new Uint8Array([10, 20, 30]);
console.log(buffer); // Uint8Array(3) [10, 20, 30]
```

---

১০. Array from Set / Map

কখন কাজে লাগে: Unique value বা key-value pair থেকে array বানাতে।

Syntax:

```
const arrayName = [...new Set(iterable)];
const arrayName = [...mapName.entries()];
```

Example:

```
const unique = [...new Set([1, 2, 2, 3, 3])]; // [1, 2, 3]

const map = new Map([["a", 1], ["b", 2]]);
const entries = [...map.entries()]; // [["a",1], ["b",2]]
```

JavaScript Loop লেখার প্রকারভেদ

Loop লেখার ২০টি উপায়

1. for — `for (let i=0; i<n; i++)`
2. while — `while (condition)`
3. do...while — `do {} while (condition)`
4. for...of — `for (const x of arr)`
5. for...in — `for (const k in obj)`
6. forEach() — `arr.forEach(cb)`
7. map() — `arr.map(cb)`

8. filter() — `arr.filter(cb)`
9. reduce() — `arr.reduce(cb, init)`
10. some() — `arr.some(cb)`
11. every() — `arr.every(cb)`
12. find() — `arr.find(cb)`
13. findIndex() — `arr.findIndex(cb)`
14. break — `if (x) break`
15. continue — `if (x) continue`
16. Nested Loop — `for { for { } }`
17. Labeled Loop — `outer: for { }`
18. Recursion — `fn() { fn() }`
19. for await...of — `for await (const x of iter)`
20. entries() loop — `for (const [i, v] of arr.entries())`

Must Know ৭টি:

1. `for` — বেসিক loop
2. `for...of` — Array value
3. `for...in` — Object key
4. `forEach()` — Side effects
5. `map()` — Transform
6. `filter()` — Select
7. `reduce()` — Aggregate

Tip: Frontend-এ `map`, `filter`, `reduce`, `for...of` — এই চারটি সবচেয়ে বেশি ব্যবহৃত হয়। React-এ JSX render করার সময় `map` প্রায় প্রতিদিনই লাগবে।

১. for Loop (Classic)

কখন কাজে লাগে: নির্দিষ্ট সংখ্যক বার repeat করতে, index দরকার হলে।

Syntax:

```
for (initialization; condition; increment) {
  // code
}
```

Example:

```
for (let i = 1; i <= 5; i++) {
  console.log(i); // 1 2 3 4 5
}
```

---

২. while Loop

কখন কাজে লাগে: Condition সত্য থাকা পর্যন্ত চলবে, কতবার হবে আগে থেকে জানা না থাকলে।

Syntax:

```
while (condition) {
  // code
```

```
  // increment/decrement
}
```

Example:

```
let i = 1;
while (i <= 5) {
  console.log(i); // 1 2 3 4 5
  i++;
}
```

---

৩. do...while Loop

কখন কাজে লাগে: অন্তত একবার code চালাতে হবে, তারপর condition check।

Syntax:

```
do {
  // code
} while (condition);
```

Example:

```
let i = 1;
do {
  console.log(i); // 1 2 3 4 5
  i++;
} while (i <= 5);
```

---

৪. for...of Loop (ES6)

কখন কাজে লাগে: Array, string, Map, Set-এর value iterate করতে।

Syntax:

```
for (const item of iterable) {
  // code
}
```

Example:

```
const fruits = ["apple", "banana", "mango"];
for (const fruit of fruits) {
  console.log(fruit); // apple banana mango
}
```

---

৫. for...in Loop

কখন কাজে লাগে: Object-এর key/property iterate করতে।

Syntax:

```
for (const key in object) {
  // code
}
```

Example:

```
const user = { name: "Rahim", age: 25 };
for (const key in user) {
  console.log(key, user[key]); // name Rahim, age 25
}
```

---

৬. forEach() Method

কখন কাজে লাগে: Array-এর প্রতিটি element-এ কাজ করতে (side effects)।

Syntax:

```
array.forEach((item, index, array) => {
  // code
});
```

Example:

```
[10, 20, 30].forEach((num, i) => {
  console.log(i, num); // 0 10, 1 20, 2 30
});
```

---

৭. map() Method (Loop-এর মতো)

কখন কাজে লাগে: প্রতিটি element transform করে নতুন array বানাতে।

Syntax:

```
const newArray = array.map((item, index) => {
  return transformedItem;
});
```

Example:

```
const doubled = [1, 2, 3].map(n => n * 2);
console.log(doubled); // [2, 4, 6]
```

---

৮. filter() Method

কখন কাজে লাগে: শর্ত অনুযায়ী element বাছাই করতে।

Syntax:

```
const newArray = array.filter((item) => condition);
```

Example:

```
const evens = [1, 2, 3, 4, 5].filter(n => n % 2 === 0);
console.log(evens); // [2, 4]
```

---

৯. reduce() Method

কখন কাজে লাগে: Array-কে একটি মানে পরিণত করতে (sum, max, group)।

Syntax:

```
const result = array.reduce((accumulator, item) => {
  return updatedAccumulator;
}, initialValue);
```

Example:

```
const sum = [1, 2, 3, 4].reduce((acc, n) => acc + n, 0);
console.log(sum); // 10
```

---

১০. some() Method

কখন কাজে লাগে: অন্তত একটি element শর্ত পূরণ করে কি না check করতে।

Syntax:

```
const result = array.some((item) => condition);
```

Example:

```
const hasEven = [1, 3, 4].some(n => n % 2 === 0);
console.log(hasEven); // true
```

---

১১. every() Method

কখন কাজে লাগে: সব element শর্ত পূরণ করে কি না check করতে।

Syntax:

```
const result = array.every((item) => condition);
```

Example:

```
const allPositive = [1, 2, 3].every(n => n > 0);
console.log(allPositive); // true
```

---

১২. find() Method

কখন কাজে লাগে: শর্ত মেলে এমন প্রথম element খুঁজতে।

Syntax:

```
const item = array.find((item) => condition);
```

Example:
```
const found = [1, 2, 3, 4].find(n => n > 2);
console.log(found); // 3
```

---

১৩. findIndex() Method

কখন কাজে লাগে: শর্ত মেলে এমন প্রথম element-এর index খুঁজতে।

Syntax:
```
const index = array.findIndex((item) => condition);
```

Example:
```
const idx = [1, 2, 3, 4].findIndex(n => n > 2);
console.log(idx); // 2
```

---

১৪. break Statement

কখন কাজে লাগে: Loop থেকে সাথে সাথে বের হতে।

Syntax:
```
for (...) {
  if (condition) break;
}
```

Example:
```
for (let i = 1; i <= 10; i++) {
  if (i === 5) break;
  console.log(i); // 1 2 3 4
}
```

---

১৫. continue Statement

কখন কাজে লাগে: নির্দিষ্ট iteration skip করতে।

Syntax:
```
for (...) {
  if (condition) continue;
  // code
}
```

Example:
```
for (let i = 1; i <= 5; i++) {
  if (i === 3) continue;
  console.log(i); // 1 2 4 5
```

```
}
```

---

১৬. Nested Loop

কখন কাজে লাগে: Matrix, table, pattern print করতে।

Syntax:

```
for (let i = 0; i < rows; i++) {
  for (let j = 0; j < cols; j++) {
    // code
  }
}
```

Example:

```
for (let i = 1; i <= 3; i++) {
  let row = "";
  for (let j = 1; j <= 3; j++) {
    row += `${i * j} `;
  }
  console.log(row);
}
// 1 2 3
// 2 4 6
// 3 6 9
```

---

১৭. Labeled Loop

কখন কাজে লাগে: Nested loop-এ বাইরের loop control করতে।

Syntax:

```
outerLabel: for (...) {
  for (...) {
    break outerLabel;
    continue outerLabel;
  }
}
```

Example:

```
outer: for (let i = 1; i <= 3; i++) {
  for (let j = 1; j <= 3; j++) {
    if (i === 2 && j === 2) break outer;
    console.log(i, j);
  }
}
```

---

১৮. Recursion (Loop-এর বিকল্প)

কখন কাজে লাগে: Tree traversal, factorial, nested data handle করতে।

Syntax:

```
function functionName(n) {
  if (baseCondition) return;
  // code
  functionName(n - 1);
}
```

Example:

```
function factorial(n) {
  if (n <= 1) return 1;
  return n * factorial(n - 1);
}
console.log(factorial(5)); // 120
```

---

১৯. for await...of (Async Loop)

কখন কাজে লাগে: Async iterable (stream, fetch data) loop করতে।

Syntax:

```
for await (const item of asyncIterable) {
  // code
}
```

Example:

```
async function process() {
  const promises = [Promise.resolve(1), Promise.resolve(2)];
  for await (const val of promises) {
    console.log(val); // 1 2
  }
}
process();
```

---

২০. for...of with entries()

কখন কাজে লাগে: Array-এর index + value একসাথে দরকার হলে।

Syntax:

```
for (const [index, value] of array.entries()) {
  // code
}
```

Example:

```
for (const [i, fruit] of ["apple", "banana"].entries()) {
  console.log(i, fruit); // 0 apple, 1 banana
}
```

JavaScript Condition লেখার প্রকারভেদ

Condition লেখার ২০টি উপায়

1. if — `if (cond) {}`
2. if...else — `if (c) {} else {}`
3. else if — `if (c1) {} else if (c2) {}`
4. Nested if — `if (c1) { if (c2) {} }`
5. Ternary — `c ? a : b`
6. Nested Ternary — `c1 ? a : c2 ? b : d`
7. switch — `switch (x) { case: }`
8. Fall-through switch — `case a: case b:`
9. && short-circuit — `c && expr`
10. || short-circuit — `a || b`
11. ?? nullish — `a ?? b`
12. ?. optional chaining — `obj?.prop`
13. Logical assignment — `&&=, ||=, ??=`
14. Object lookup — `map[key]`
15. includes() — `arr.includes(x)`
16. Truthy/Falsy — `if (value)`
17. try...catch — `try {} catch {}`
18. Guard clause — `if (!x) return;`
19. Conditional fn call — `c ? fnA() : fnB()`
20. switch(true) — `switch (true) { case c: }`

Must Know ৮টি:

1. `if...else` / `else if`
2. `Ternary ? :`
3. `&&` / `||` short-circuit
4. `??` nullish coalescing
5. `?.` optional chaining
6. Truthy / Falsy
7. `switch`
8. `try...catch`

Tip: React-এ JSX-এর ভেতরে `&&`, ternary, এবং `?.` — এই তিনটি সবচেয়ে বেশি ব্যবহৃত হয়। `??` এবং `||`-এর পার্থক্য ভালোভাবে বুঝে রাখুন — এটা interview-এ common প্রশ্ন।

১. if Statement

কখন কাজে লাগে: একটি শর্ত সত্য হলে code চালাতে।

Syntax:

```
if (condition) {
  // code
}
```

Example:

```
const age = 20;
if (age >= 18) {
```

```
  console.log("Adult");
}
```

---

২. if...else Statement

কখন কাজে লাগে: শর্ত সত্য হলে এক কাজ, মিথ্যা হলে অন্য কাজ।

Syntax:

```
if (condition) {
  // code
} else {
  // code
}
```

Example:

```
const age = 15;
if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}
```

---

৩. if...else if...else Statement

কখন কাজে লাগে: একাধিক শর্ত পরীক্ষা করতে।

Syntax:

```
if (condition1) {
  // code
} else if (condition2) {
  // code
} else {
  // code
}
```

Example:

```
const score = 75;
if (score >= 80) {
  console.log("A+");
} else if (score >= 70) {
  console.log("A");
} else if (score >= 60) {
  console.log("B");
} else {
  console.log("Fail");
}
```

---

৪. Nested if

কখন কাজে লাগে: একটি শর্তের ভেতরে আরেকটি শর্ত পরীক্ষা করতে।

Syntax:

```
if (condition1) {
  if (condition2) {
    // code
  }
}
```

Example:

```
const age = 25;
const hasLicense = true;

if (age >= 18) {
  if (hasLicense) {
    console.log("Can drive");
  }
}
```

---

৫. Ternary Operator (?:)

কখন কাজে লাগে: Short if-else, এক লাইনে condition লিখতে।

Syntax:

```
condition ? expressionIfTrue : expressionIfFalse;
```

Example:

```
const age = 20;
const status = age >= 18 ? "Adult" : "Minor";
console.log(status); // Adult
```

---

৬. Nested Ternary

কখন কাজে লাগে: একাধিক condition এক লাইনে (কিন্তু readability খারাপ হয়)।

Syntax:

```
condition1 ? value1 : condition2 ? value2 : value3;
```

Example:

```
const score = 75;
const grade = score >= 80 ? "A+" : score >= 70 ? "A" : "B";
console.log(grade); // A
```

---

৭. switch Statement

কখন কাজে লাগে: একটি variable-এর অনেকগুলো নির্দিষ্ট মানের মধ্যে তুলনা করতে।

Syntax:

```
switch (expression) {
  case value1:
    // code
    break;
  case value2:
    // code
    break;
  default:
    // code
}
```

Example:

```
const day = "Monday";
switch (day) {
  case "Monday":
    console.log("Start of week");
    break;
  case "Friday":
    console.log("Weekend coming");
    break;
  default:
    console.log("Regular day");
}
```

---

৮. switch with Fall-through

কখন কাজে লাগে: একাধিক case-এ একই code চালাতে (break না দিয়ে)।

Syntax:

```
switch (expression) {
  case value1:
  case value2:
    // code
    break;
}
```

Example:

```
const fruit = "mango";
switch (fruit) {
  case "apple":
  case "mango":
  case "banana":
    console.log("This is a fruit");
    break;
  default:
```

```
    console.log("Unknown");
}
```

---

৯. Logical AND (&&) Short-circuit

কখন কাজে লাগে: Condition সত্য হলে কিছু execute করতে (React-এ খুব common)।

Syntax:
```
condition && expression;
```

Example:
```
const isLoggedIn = true;
isLoggedIn && console.log("Welcome!"); // Welcome!

// React style
// {isLoggedIn && <Dashboard />}
```

---

১০. Logical OR (||) Short-circuit

কখন কাজে লাগে: Default value সেট করতে, প্রথম truthy value নিতে।

Syntax:
```
value1 || value2;
```

Example:
```
const username = "" || "Guest";
console.log(username); // Guest
```

---

১১. Nullish Coalescing (??)

কখন কাজে লাগে: শুধু `null` বা `undefined` হলে default দিতে (`0`, `""` valid ধরতে)।

Syntax:
```
value ?? defaultValue;
```

Example:
```
const count = 0 ?? 10;
console.log(count); // 0  (|| হলে হতো 10)

const name = null ?? "Guest";
console.log(name); // Guest
```

---

১২. Optional Chaining (?.)

কখন কাজে লাগে: Nested object property safely access করতে (error এড়াতে)।

Syntax:
```
object?.property;
object?.method?.();
object?.[key];
```

Example:
```
const user = { name: "Rahim" };
console.log(user.address?.city); // undefined (error না)
```

```
const users = [{ name: "A" }];
console.log(users[0]?.name); // A
```

---

১৩. Logical Assignment Operators

কখন কাজে লাগে: Condition সহ value assign করতে।

Syntax:

```
a &&= b;   // a truthy হলে assign
a ||= b;   // a falsy হলে assign
a ??= b;   // a null/undefined হলে assign
```

Example:

```
let a = 1;
a &&= 5;
console.log(a); // 5

let b = null;
b ??= 10;
console.log(b); // 10

let c = 0;
c ||= 7;
console.log(c); // 7
```

---

১৪. Object Lookup (Switch-এর বিকল্প)

কখন কাজে লাগে: Switch-এর চেয়ে clean way-তে condition map করতে।

Syntax:

```
const map = {
  key1: value1,
  key2: value2
};
const result = map[key] || defaultValue;
```

Example:

```
const dayType = {
  Monday: "Start",
  Friday: "Weekend",
  Sunday: "Holiday"
};
console.log(dayType["Friday"] || "Regular"); // Weekend
```

---

১৫. Array.includes() as Condition

কখন কাজে লাগে: Value একটি list-এর মধ্যে আছে কি না check করতে।

Syntax:

```
array.includes(value);
```

Example:

```
const roles = ["admin", "editor"];
```

```
if (roles.includes("admin")) {
  console.log("Has access");
}
```

---

১৬. Truthy / Falsy Check

কখন কাজে লাগে: Value সত্য/মিথ্যা কি না সরাসরি check করতে।

Syntax:

```
if (value) {
  // truthy
}
```

Falsy values: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`

Example:

```
const name = "";
if (name) {
  console.log("Has name");
} else {
  console.log("Empty"); // Empty
}
```

---

১৭. try...catch as Condition (Error handling)

কখন কাজে লাগে: Runtime error handle করতে (condition দিয়ে সম্ভব নয়)।

Syntax:

```
try {
  // risky code
} catch (error) {
  // handle
} finally {
  // always runs
}
```

Example:

```
try {
  JSON.parse("invalid json");
} catch (e) {
  console.log("Parse error:", e.message);
}
```

---

১৮. Guard Clause (Early Return)

কখন কাজে লাগে: Nested if কমিয়ে clean code লিখতে।

Syntax:

```
function fn(value) {
  if (!value) return;
  // main logic
}
```

Example:

```
function greet(user) {
  if (!user) return "No user";
  if (!user.name) return "No name";
  return "Hello " + user.name;
}
console.log(greet({ name: "Rahim" })); // Hello Rahim
```

---

১৯. Conditional Function Call

কখন কাজে লাগে: Condition অনুযায়ী আলাদা function call করতে।

Syntax:

```
condition ? fnA() : fnB();
```

Example:

```
const isAdmin = true;
isAdmin ? showAdminPanel() : showUserPanel();

function showAdminPanel() { console.log("Admin"); }
```

```
function showUserPanel() { console.log("User"); }
```

---

২০. Pattern Matching (switch true)

কখন কাজে লাগে: switch-এ condition (range) ব্যবহার করতে।

Syntax:

```
switch (true) {
  case condition1:
    // code
    break;
  case condition2:
    // code
    break;
}
```

Example:

```
const score = 75;
switch (true) {
  case score >= 80:
    console.log("A+");
    break;
  case score >= 70:
    console.log("A"); // A
    break;
  default:
    console.log("B");
}
```

JavaScript This লেখার প্রকারভেদ

`this` কী?

`this` হলো একটি special keyword যা runtime-এ determine হয় অর্থাৎ কোন context-এ function call হচ্ছে, তার উপর নির্ভর করে `this`-এর মান বদলায়।
মূল কথা: `this` কোনো fixed value নয়। এটি একটি reference যা current execution context-কে point করে।
সবচেয়ে গুরুত্বপূর্ণ নিয়ম: `this` নির্ধারিত হয় কীভাবে function call করা হলো তার উপর কোথায় define করা হলো তার উপর নয়। (Arrow function ব্যতিক্রম)

`this` কী কাজ করে:

- একই function অনেক object-এ reuse করা যায়
- Method কে তার নিজের object-এর data access করতে দেয়
- Inheritance-এ parent method child-এর data ব্যবহার করতে পারে

```
const user1 = {
  name: "Rahim",
  greet: function() {
    console.log("Hello, " + user1.name);
  }
};

const user2 = {
  name: "Karim",
  greet: function() {
    console.log("Hello, " + user1.name); // ✗ ভুল নাম আসবে
  }
};
user2.greet(); // "Hello, Rahim" — Karim নয়!
```

উপরের code-এ প্রতিটি object-এর জন্য আলাদা function লিখতে হবে। `this` এই সমস্যার সমাধান:

```
const user1 = {
  name: "Rahim",
  greet: function() {
    console.log("Hello, " + this.name); // ☑ dynamic
  }
};

const user2 = {
  name: "Karim",
  greet: user1.greet // একই function reuse
};
user1.greet(); // "Hello, Rahim"
user2.greet(); // "Hello, Karim"
```

`this` কীভাবে determine হয় — ৬টি Rule

Rule ১: Global Context

কোথায়: Function-এর বাইরে, বা সাধারণ function call-এ (non-strict mode)।

Syntax:

```
console.log(this);
```

Example:

```
console.log(this); // Browser-এ: window, Node-এ: {}
function show() {
  console.log(this); // Browser-এ: window (non-strict)
}
show();
```

Strict mode-এ:

```
"use strict";
function show() {
  console.log(this); // undefined
}
show();
```

Rule ২: Object Method (Implicit Binding)

কোথায়: যখন function কোনো object-এর method হিসেবে call হয়।

Syntax:

```
object.method();
// this = object
const person = {
  name: "Rahim",
  greet: function() {
    console.log(this.name);
  }
};
person.greet(); // "Rahim" — this = person
```

Nested object-এ সাবধান:

```
const obj = {
  name: "Outer",
  inner: {
    name: "Inner",
    show: function() {
      console.log(this.name);
    }
  }
};

obj.inner.show(); // "Inner" — this = obj.inner (যে object দিয়ে call)
```

Method আলাদা করলে `this` হারায়:

```
const person = {
  name: "Rahim",
  greet: function() {
    console.log(this.name);
  }
};

const fn = person.greet; // reference copy
fn(); // undefined — this = window/undefined
```

⚠ **এটাই সবচেয়ে common bug!** Callback-এ method পাঠালে `this` হারিয়ে যায়।

Rule ৩: Explicit Binding (call, apply, bind)

কোথায়: যখন আপনি নিজে `this` নির্ধারণ করেন।

Syntax:

```
functionName.call(thisArg, arg1, arg2);
functionName.apply(thisArg, [arg1, arg2]);
const boundFn = functionName.bind(thisArg);
```

Example:

```
function greet() {
  console.log("Hello, " + this.name);
}

const user = { name: "Rahim" };

greet.call(user);              // "Hello, Rahim"
greet.apply(user);             // "Hello, Rahim"

const bound = greet.bind(user);
bound();                       // "Hello, Rahim"
```

Arguments সহ:

```
function introduce(age, city) {
  console.log(`${this.name}, ${age}, ${city}`);
}

const user = { name: "Rahim" };

introduce.call(user, 25, "Dhaka");        // Rahim, 25, Dhaka
introduce.apply(user, [25, "Dhaka"]);     // Rahim, 25, Dhaka

const bound = introduce.bind(user, 25);
bound("Dhaka");                           // Rahim, 25, Dhaka
```

কোনটা কখন:

Method	কখন
`call()`	Arguments আলাদা আলাদা
`apply()`	Arguments array-তে
`bind()`	Permanent `this` bind, পরে call করার জন্য

Rule ৪: Constructor Function (new Binding)

কোথায়: যখন `new` দিয়ে function call হয়।

Syntax:

```
const instance = new FunctionName();
// this = নতুন তৈরি হওয়া object
```

Example:

```
function Person(name, age) {
  this.name = name;
  this.age = age;
  console.log(this); // Person { name: "Rahim", age: 25 }
}
```

```
const user = new Person("Rahim", 25);
console.log(user.name); // "Rahim"
```

`new` কী করে:

1. নতুন empty object তৈরি করে
2. `this` কে সেই object-এ point করে
3. `this`-এ property assign করে
4. object return করে

Rule ৫: Arrow Function (Lexical this)

কোথায়: Arrow function-এ `this` তার **parent scope** থেকে inherit করে।

Syntax:

```
const fn = () => {
  // this = parent scope-এর this
};
```

Example:

```
const person = {
  name: "Rahim",

  greetRegular: function() {
    console.log(this.name); // Rahim
  },

  greetArrow: () => {
    console.log(this.name); // undefined — this = global
  }
};

person.greetRegular(); // Rahim
person.greetArrow();   // undefined
```

Arrow function-এর সুবিধা (callback-এ):

```
const counter = {
  count: 0,

  startBad: function() {
    setInterval(function() {
      this.count++; // ✗ this = window
      console.log(this.count); // NaN
    }, 1000);
  },

  startGood: function() {
    setInterval(() => {
      this.count++; // ☑ this = counter
      console.log(this.count);
    }, 1000);
  }
};
```

🔑 Arrow function-এ `this` override করা যায় না — `call`, `apply`, `bind` কাজ করে না।

Rule ৬: Event Handler-এ this

কোথায়: DOM event listener-এ `this` = যে element-এ listener বসানো।

Syntax:

```
element.addEventListener("click", function() {
  console.log(this); // element
});
```

Example:

```
const btn = document.querySelector("button");

btn.addEventListener("click", function() {
  console.log(this); // <button> element
  this.style.color = "red";
});

// Arrow হলে this = window ❌
btn.addEventListener("click", () => {
  console.log(this); // window
});
```

---

this না দিলে কী হবে? বিকল্প কী?

বিকল্প ১: Hardcoded Object Name

`this` ছাড়া:

```
const person = {
  name: "Rahim",
  greet: function() {
    console.log("Hello, " + person.name); // ❌ hardcoded
  }
};
person.greet(); // "Hello, Rahim"
```

সমস্যা:

```
const person2 = {
  name: "Karim",
  greet: person.greet
};
person2.greet(); // "Hello, Rahim" ❌ — Karim হওয়া উচিত
```

👉 `this` না দিলে reuse করা যায় না।

বিকল্প ২: Closure Variable

```
function createPerson(name) {
  return {
    name: name,
    greet: function() {
      console.log("Hello, " + name); // closure
    }
  };
}

const p1 = createPerson("Rahim");
const p2 = createPerson("Karim");
p1.greet(); // "Hello, Rahim"
p2.greet(); // "Hello, Karim"
```

👉 এটা কাজ করে, কিন্তু প্রতিটি object-এর জন্য নতুন function তৈরি হয় — memory waste।

বিকল্প ৩: Function Parameter

```
function greet(self) {
  console.log("Hello, " + self.name);
}

const person = { name: "Rahim" };
greet(person); // প্রতিবার object পাঠাতে হবে
```

👉 এটা ঠিক `this`-এর মতোই কাজ করে ,আসলে `this` হলো implicit parameter যা automatically পাঠানো হয়।

বিকল্প ৪: Class / Modern Approach

```
class Person {
  constructor(name) {
    this.name = name;
  }
  greet() {
    console.log("Hello, " + this.name);
  }
}

const p = new Person("Rahim");
```

```
p.greet(); // "Hello, Rahim"
```

👉 Class-এর ভেতরে `this` ছাড়া উপায় নেই।

---

this না দিলে কী কী সমস্যা হয়?

সমস্যা ১: Object-এর নিজের data access করা যায় না

```
const user = {
  name: "Rahim",
  showName: function() {
    console.log(name); // ❌ ReferenceError
  }
};
```

সমস্যা ২: Method reuse করা যায় না

একই method দুই object-এ চালালে ভুল data আসবে।

সমস্যা ৩: Callback-এ reference হারায়

```
const user = {
  name: "Rahim",
  greet: function() {
    console.log(this.name);
  }
};

setTimeout(user.greet, 1000); // undefined — this হারিয়ে গেছে

// সমাধান:
setTimeout(user.greet.bind(user), 1000); // Rahim
setTimeout(() => user.greet(), 1000);    // Rahim
```

সমস্যা ৪: Subclass-এ parent method কাজ করে না

```
class Animal {
  speak() {
    console.log(this.name + " makes sound");
  }
}

class Dog extends Animal {
  constructor(name) {
    super();
    this.name = name;
  }
}

const d = new Dog("Tommy");
d.speak(); // "Tommy makes sound" — this ছাড়া সম্ভব নয়
```

সমস্যা ৫: Dynamic property access করা যায় না

```
const obj = {
  data: [1, 2, 3],
  sum: function() {
    return this.data.reduce((a, b) => a + b, 0); // this দরকার
  }
};
```

`this` বোঝার Master Table

Call Style	`this` মান	Example
Global (non-strict)	`window` / `global`	`fn()`
Global (strict)	`undefined`	`"use strict"; fn()`
Object method	Object	`obj.fn()`
Extracted method	`undefined` / `window`	`const f = obj.fn; f()`
`call` / `apply`	প্রথম argument	`fn.call(obj)`
`bind`	Bound object (permanent)	`fn.bind(obj)()`
`new`	নতুন instance	`new Fn()`
Arrow function	Parent scope-এর `this`	`() => this`
Event listener (regular)	`element`	`el.addEventListener`
Event listener (arrow)	Parent scope	`el.addEventListener("click", () => {})`
Class method	Instance	`instance.method()`
Class arrow property	Instance	`method = () => {}`

`this` বের করার সহজ নিয়ম (Priority Order)

1. `new` দিয়ে call? → নতুন object
2. `call`/`apply`/`bind` ব্যবহার? → সেই object
3. **Object method** হিসেবে call? → সেই object
4. **Arrow function**? → parent-এর `this` (উপরের নিয়ম skip)
5. **Default** → `window` (non-strict) / `undefined` (strict)

Frontend Developer হিসেবে Must Know

Priority	Concept	কেন জানতে হবে

⭐⭐⭐⭐⭐	Method-এ `this`	Object, class
⭐⭐⭐⭐⭐	Arrow vs Regular `this`	React, callback bug
⭐⭐⭐⭐⭐	`bind()`	Event handler, callback
⭐⭐⭐⭐⭐	Lost `this` problem	সবচেয়ে common bug
⭐⭐⭐⭐⭐	`this` in event handler	DOM manipulation
⭐⭐⭐⭐	`call` / `apply`	Function borrowing
⭐⭐⭐⭐	`new` binding	Constructor, class
⭐⭐⭐⭐	Strict mode-এ `this`	ES modules
⭐⭐⭐	Global `this`	Browser vs Node
⭐⭐⭐	Nested object `this`	Complex data

সারসংক্ষেপ

`this` কী:

- Runtime-এ determine হয়
- Function কীভাবে call হলো তার উপর নির্ভর করে
- Arrow function ব্যতিক্রম — lexical `this`

কেন দরকার:

- Object-এর নিজের data access
- Method reuse
- Inheritance
- Dynamic binding

বিকল্প কী:

- Hardcoded object name (reuse হয় না)
- Closure (memory waste)
- Function parameter (manual কাজ)
- Class (modern approach)

`this` না দিলে সমস্যা:

- Object data access নষ্ট
- Method reuse অসম্ভব
- Callback-এ reference হারায়
- Inheritance ভাঙে
- Dynamic property access হয় না

Tip: `this` বুঝতে সবসময় প্রশ্ন করুন: "এই function টা কে call করছে?" — উত্তরই `this`। আর Arrow function দেখলে প্রশ্ন করুন: "এর parent scope-এর `this` কী?"

`return` কী কাজ করে:

- Function-এর output কে বাইরে ব্যবহারযোগ্য করে
- Function composition সম্ভব করে
- Execution flow নিয়ন্ত্রণ করে (early exit)
    - Data transformation সম্ভব করে

JavaScript Return লেখার প্রকারভেদ

`return` কী?

`return` হলো একটি statement যা:

1. Function-এর execution সাথে সাথে বন্ধ করে দেয়
2. Function থেকে একটি value caller-এর কাছে পাঠায় (ফেরত দেয়)
3. এর পরের কোনো code আর চলে না

মূল কথা: Function হলো একটি machine — input নেয়, কাজ করে, output দেয়। `return` হলো সেই output দেওয়ার দরজা।

🔑 সবচেয়ে গুরুত্বপূর্ণ নিয়ম: `return` না থাকলে function `undefined` return করে (by default)।

কেন `return` দরকার?

```
function add(a, b) {
  console.log(a + b); // শুধু print করে
}

const result = add(5, 3); // print: 8
console.log(result);      // undefined ✗
```

Function কাজ করছে, কিন্তু ফলাফল বাইরে ব্যবহার করা যাচ্ছে না।

`return` দিলে:

```
function add(a, b) {
  return a + b; // value ফেরত দেয়
}

const result = add(5, 3);
console.log(result);      // 8 ☑
console.log(result * 2);  // 16 ☑
```

`return` কীভাবে কাজ করে — ৬টি Rule

Rule ১: Value Return করা

কোথায়: যখন function থেকে value বাইরে পাঠাতে হবে।

Syntax:

```
return value;
```

Example:

```
function square(n) {
  return n * n;
```

```
}

console.log(square(5)); // 25
```

Rule ২: `return` না দিলে `undefined` Return হয়

কোথায়: যখন function-এর কোনো explicit return নেই।

Syntax:

```
function fn() {
  // return নেই
}
// fn() === undefined
```

Example:

```
function greet(name) {
  console.log("Hello " + name);
}

const result = greet("Rahim");
console.log(result); // undefined
```

👉 Function কিছু print করলেও return value নেই।

Rule ৩: Early Return (Guard Clause)

কোথায়: Condition মিথ্যা হলে দ্রুত function থেকে বের হতে।

Syntax:

```
function fn(value) {
  if (!value) return;
  // main logic
}
```

Example:

```
function divide(a, b) {
  if (b === 0) return "Cannot divide by zero";
  return a / b;
}

console.log(divide(10, 2)); // 5
console.log(divide(10, 0)); // "Cannot divide by zero"
```

Nested if থেকে clean:

```
// ❌ Nested if
```

```
function getGrade(score) {
  if (score >= 0) {
    if (score <= 100) {
      if (score >= 80) {
        return "A+";
      }
    }
  }
  return "Invalid";
}

// ☑ Guard clause
function getGrade(score) {
  if (score < 0 || score > 100) return "Invalid";
  if (score >= 80) return "A+";
  if (score >= 70) return "A";
  return "B";
}
```

Rule 8: Multiple Value Return (Object/Array)

কোথায়: একসাথে অনেক value return করতে।

Syntax:

```
return { key1, key2 };
return [value1, value2];
```

Example:

```
function getMinMax(arr) {
  return {
    min: Math.min(...arr),
    max: Math.max(...arr)
  };
}

const { min, max } = getMinMax([3, 1, 7, 2]);
console.log(min, max); // 1 7

// Array version
function getCoords() {
  return [10, 20];
}
const [x, y] = getCoords();
```

Rule ৫: Function Return করা (Higher-Order)

কোথায়: Closure তৈরি করতে, currying-এ।

Syntax:
```
return function() { };
return (x) => x;
```

Example:
```
function multiplier(factor) {
  return function(number) {
    return number * factor;
  };
}

const double = multiplier(2);
console.log(double(5)); // 10

const triple = multiplier(3);
console.log(triple(5)); // 15
```

Rule ৬: `return` Execution থামায়

কোথায়: `return`-এর পরের code কখনো চলে না।

Syntax:
```
return value;
// এই লাইনের পরের code dead code
```

Example:
```
function test() {
  console.log("1");
  return "done";
  console.log("2"); // ✗ কখনো চলবে না
}

console.log(test());
// Output: "1", "done"
```

Loop-এ return:
```
function findFirst(arr, target) {
  for (const item of arr) {
    if (item === target) return item; // সাথে সাথে বের
  }
  return null;
}
console.log(findFirst([1, 2, 3], 2)); // 2
```

`return` VS `console.log` — বড় পার্থক্য

```
// console.log — শুধু দেখায়
function addLog(a, b) {
  console.log(a + b); // screen-এ print
}

// return — value ফেরত দেয়
function addReturn(a, b) {
  return a + b; // caller ব্যবহার করতে পারে
}

const x = addLog(2, 3);        // print: 5
console.log(x);                // undefined

const y = addReturn(2, 3);     // কিছু print হয় না
console.log(y);                // 5
console.log(y * 10);           // 50
```

👉 `console.log` = debug, `return` = real output

`return` না দিলে কী হবে? বিকল্প কী?

বিকল্প ১: Global Variable-এ Store করা

```
let result; // global

function add(a, b) {
  result = a + b; // global-এ রাখা
}

add(5, 3);
console.log(result); // 8
```

সমস্যা:

- Global namespace দূষিত হয়
- Multiple call-এ overwrite হয়
- Debug কঠিন
- Race condition

বিকল্প ২: Callback-এ পাঠানো

```
function add(a, b, callback) {
  callback(a + b);
}

add(5, 3, (result) => {
```

```
  console.log(result); // 8
});
```

সমস্যা:

- Callback hell
- Synchronous value পাওয়া কঠিন
- Composition কঠিন

বিকল্প ৩: Object-এ Store করা

```
const calculator = {
  result: 0,
  add(a, b) {
    this.result = a + b;
  }
};

calculator.add(5, 3);
console.log(calculator.result); // 8
```

সমস্যা:

- State management জটিল
- Pure function নয়
- Test কঠিন

তুলনা: `return` কেন সেরা?

পদ্ধতি	Value পাওয়া	Reuse	Test	Pure
`return`	✅ Direct	✅	✅	✅
Global var	❌ Side effect	❌	❌	❌
Callback	⚠️ Async	✅	⚠️	⚠️
Object store	⚠️ State	⚠️	⚠️	❌

`return` না দিলে কী কী সমস্যা হয়?

সমস্যা ১: Value ব্যবহার করা যায় না

```
function getTotal(prices) {
  prices.reduce((a, b) => a + b, 0); // ❌ return নেই
}

const total = getTotal([10, 20, 30]);
console.log(total); // undefined
```

✅ সমাধান:

```
function getTotal(prices) {
  return prices.reduce((a, b) => a + b, 0); // ✅
}
```

সমস্যা ২: Chain করা যায় না

```
function double(n) {
  n * 2; // ❌ return নেই
}

// Chain করা যাবে না
double(double(5)); // undefined
```

✅ সমাধান:

```
function double(n) {
  return n * 2;
}
console.log(double(double(5))); // 20
```

সমস্যা ৩: Array Method-এ ভুল ফলাফল

```
const nums = [1, 2, 3];

// ❌ Arrow function-এ {} দিলে return লাগে
const doubled = nums.map(n => { n * 2; });
console.log(doubled); // [undefined, undefined, undefined]

// ✅ সমাধান ১: Implicit return
const doubled1 = nums.map(n => n * 2);

// ✅ সমাধান ২: Explicit return
const doubled2 = nums.map(n => { return n * 2; });
```

👉 সবচেয়ে common bug!

সমস্যা ৪: Function Composition ভাঙে

```
const pipe = (...fns) => (x) => fns.reduce((v, fn) => fn(v), x);

const addOne = x => { x + 1; };       // ❌ return নেই
const double = x => x * 2;

console.log(pipe(addOne, double)(5)); // NaN
```

সমস্যা ৫: Async Function-এ Value হারায়

```
async function fetchUser() {
  const res = await fetch("/api/user");
  res.json(); // ❌ return নেই
}

const user = await fetchUser();
console.log(user); // undefined
```

✅ সমাধান:

```
async function fetchUser() {
  const res = await fetch("/api/user");
  return await res.json();
}
```

সমস্যা ৬: Condition-এ ভুল Branch

```
function checkAge(age) {
  if (age >= 18) {
    "Adult"; // ❌ return নেই
  } else {
    "Minor";
  }
}

console.log(checkAge(20)); // undefined
```

✅ সমাধান:

```
function checkAge(age) {
  return age >= 18 ? "Adult" : "Minor";
}
```

সমস্যা ৭: Recursion ভাঙে

```
function factorial(n) {
  if (n <= 1) return 1;
  n * factorial(n - 1); // ❌ return নেই
}

console.log(factorial(5)); // undefined
```

✅ সমাধান:

```
function factorial(n) {
  if (n <= 1) return 1;
  return n * factorial(n - 1); // ✅
}
```

Implicit vs Explicit Return

Explicit Return

```
function fn() {
  return value; // স্পষ্টভাবে লেখা
}

const arrow = () => {
  return value; // { } থাকলে explicit লাগে
};
```

Implicit Return (শুধু Arrow Function)

```
const arrow = () => value;          // এক লাইন
const arrow2 = () => ({ key: 1 });  // object wrap করতে () লাগে
const arrow3 = (a, b) => a + b;
```

⚠ সাবধান:

```
const fn = () => { value: 1 }; // ✗ এটা object return নয়
// {} কে block ধরে, তাই undefined return করে
const fn2 = () => ({ value: 1 }); // ☑ object return
```

`return`-এর বিশেষ ব্যবহার

১. Void Function (return কিছু নেই)

```
function logMessage(msg) {
  console.log(msg);
  return; // optional, undefined return করে
}
```

২. `return` এরপর newline সাবধান

```
function fn() {
  return
  {
    value: 1
  };
}
console.log(fn()); // undefined ✗ (ASI - Automatic Semicolon Insertion)

// ☑ সঠিক
function fn2() {
  return {
    value: 1
  };
}
```

৩. Constructor-এ `return`

```
function Person(name) {
  this.name = name;
  return { other: "object" }; // ⚠️ Object return করলে this বাদ পড়ে
}
console.log(new Person("Rahim")); // { other: "object" }

// Primitive return করলে ignore হয়
function Person2(name) {
  this.name = name;
  return 42; // ignore
}
console.log(new Person2("Karim")); // Person2 { name: "Karim" }
```

৪. Async Function-এ `return`

```
async function getData() {
  return 42; // Promise.resolve(42) হিসেবে wrap হয়
}

getData().then(v => console.log(v)); // 42
```

৫. Generator-এ `return`

```
function* gen() {
  yield 1;
  yield 2;
  return "done"; // { value: "done", done: true }
}

const g = gen();
console.log(g.next()); // { value: 1, done: false }
console.log(g.next()); // { value: 2, done: false }
console.log(g.next()); // { value: "done", done: true }
```

`return` Master Table

Situation	Return কী	Output
`return value;`	Value ফেরত	value
`return;`	খালি return	undefined
কোনো return নেই	Default	undefined
`return` + newline	ASI	undefined
Arrow `() => x`	Implicit	x
Arrow `() => { x }`	Block	undefined ❌

Arrow `() => ({x})`	Object wrap	{x}
`async return x`	Wrapped	Promise(x)
`yield` + `return`	Generator end	{value, done:true}
Constructor `return obj`	Override	obj
Constructor `return prim`	Ignored	this

বাস্তব উদাহরণ — সব Rule একসাথে

```
// 1. Basic return
function add(a, b) {
  return a + b;
}
// 2. Early return (guard)
function greet(user) {
  if (!user) return "No user";
  if (!user.name) return "No name";
  return `Hello ${user.name}`;
}
// 3. Multiple values
function stats(arr) {
  return {
    sum: arr.reduce((a, b) => a + b, 0),
    avg: arr.reduce((a, b) => a + b, 0) / arr.length
  };
}
// 4. Higher-order return
function makeCounter() {
  let count = 0;
  return () => ++count;
}
// 5. Loop + return
function findUser(users, id) {
  for (const u of users) {
    if (u.id === id) return u;
  }
  return null;
}
// 6. Recursion
function fact(n) {
  if (n <= 1) return 1;
  return n * fact(n - 1);
}
```

Must Know

Priority	Concept	কেন জানতে হবে

⭐⭐⭐⭐⭐	Basic `return value`	সব function
⭐⭐⭐⭐⭐	Arrow implicit return	React, map/filter
⭐⭐⭐⭐⭐	`return` না দিলে `undefined`	Common bug
⭐⭐⭐⭐⭐	Early return (Guard)	Clean code
⭐⭐⭐⭐⭐	Object/Array return	Multiple values
⭐⭐⭐⭐	`return` execution থামায়	Loop, validation
⭐⭐⭐⭐	Async `return`	API calls
⭐⭐⭐⭐	Function return (HOF)	Closure, currying
⭐⭐⭐	`return` + ASI newline bug	Semicolon issue
⭐⭐⭐	Constructor `return`	Class edge case
⭐⭐	Generator `return`	Advanced async

সারসংক্ষেপ

`return` কী:

- Function থেকে value বাইরে পাঠায়
- Execution সাথে সাথে বন্ধ করে
- না থাকলে `undefined` return হয়

কেন দরকার:

- Output বাইরে ব্যবহার করতে
- Function composition
- Early exit
- Data transformation

বিকল্প কী:

- Global variable (side effect, খারাপ)
- Callback (complex)
- Object store (state management)
- `return` সেরা কারণ: pure, reusable, testable

`return` না দিলে সমস্যা:

- Value `undefined` হয়
- Chain করা যায় না
- Array method ভুল ফল দেয়
- Recursion ভাঙে
- Async data হারায়
- Function composition ভাঙে

Tip: Function লেখার সময় সবসময় প্রশ্ন করুন: "এই function কী return করবে?" — যদি উত্তর না থাকে, হয় function-টা void, নয়তো `return` ভুলে গেছেন। Arrow function-এ `{ }` দেখলেই `return` লাগবে কি না check করুন।

JavaScript Number Buildin object & Function

JS বেশ কয়েকটি বিল্ট-ইন অবজেক্ট ও ফাংশন সরবরাহ করে, যেগুলোকে কাজের ভিত্তিতে প্রধানত চারটি গ্রুপে ভাগ করা যায়:

১. `Number` Constractor ও Static method

এই গ্রুপটি সরাসরি `Number` অবজেক্টের সাথে সম্পর্কিত। এখানে দুই ধরনের জিনিস আছে: Static property (যেমন `Number.MAX_VALUE`) এবং Static method (যেমন `Number.isNaN()`)। এগুলো সবসময় `Number.` দিয়ে শুরু করে ব্যবহার করতে হয়।

Static property:

- `Number.MAX_VALUE`: সর্বোচ্চ সম্ভাব্য ধনাত্মক সংখ্যা।
- `Number.MIN_VALUE`: ক্ষুদ্রতম ধনাত্মক অশূন্য সংখ্যা।
- `Number.NaN`: "Not-a-Number" নির্দেশ করে।
- `Number.NEGATIVE_INFINITY` / `Number.POSITIVE_INFINITY`: ঋণাত্মক ও ধনাত্মক অসীম।
- `Number.EPSILON`: ১ এবং তার চেয়ে বড় নিকটতম সংখ্যার মধ্যে পার্থক্য (ফ্লোটিং-পয়েন্ট তুলনার জন্য দরকারি)।
- `Number.MAX_SAFE_INTEGER`: নিরাপদে ব্যবহারযোগ্য সর্বোচ্চ পূর্ণসংখ্যা (9007199254740991)।

Static method:

- `Number.isNaN(value)`: শুধুমাত্র `NaN`-এর জন্য `true` রিটার্ন করে, স্ট্রিং কনভার্ট করে না।
- `Number.isFinite(value)`: শুধুমাত্র সসীম সংখ্যার জন্য `true` রিটার্ন করে।
- `Number.isInteger(value)`: পূর্ণসংখ্যা কিনা পরীক্ষা করে।
- `Number.parseInt(string, radix)` / `Number.parseFloat(string)`: স্ট্রিং থেকে সংখ্যা পার্স করার জন্য।

২. `Math` নেমস্পেস অবজেক্ট

`Math` একটি বিশেষ গ্লোবাল অবজেক্ট যার সবকিছুই স্ট্যাটিক। এটি কোনো কনস্ট্রাক্টর নয়, তাই `new Math()` করা যায় না। এটি `Number` টাইপের সাথে কাজ করে, `BigInt`-এর সাথে নয়। `Math`-এর কাজ মূলত গাণিতিক গণনা ও ধ্রুবক সরবরাহ করা।

স্ট্যাটিক প্রপার্টি (গাণিতিক ধ্রুবক):

- `Math.PI` (π ≈ 3.14159)
- `Math.E` (অয়লার সংখ্যা ≈ 2.718)
- `Math.LN2`, `Math.LN10`, `Math.LOG2E`, `Math.LOG10E`, `Math.SQRT1_2`, `Math.SQRT2`

স্ট্যাটিক মেথড (গাণিতিক ফাংশন):

- **Rounding:** `Math.floor()`, `Math.ceil()`, `Math.round()`, `Math.trunc()`।
- **মৌলিক গণনা:** `Math.abs()`, `Math.pow()`, `Math.sqrt()`, `Math.cbrt()`, `Math.hypot()`।
- **ত্রিকোণমিতি:** `Math.sin()`, `Math.cos()`, `Math.tan()`, `Math.asin()`, `Math.acos()`, `Math.atan()`, `Math.atan2()` এবং হাইপারবোলিক ফাংশন।
- **Logarithm:** `Math.log()`, `Math.log10()`, `Math.log2()`, `Math.log1p()`, `Math.exp()`, `Math.expm1()`।
- **Utility:** `Math.max()`, `Math.min()`, `Math.random()`, `Math.sign()`, `Math.clz32()`, `Math.imul()`, `Math.fround()`, `Math.f16round()`।

৩. `Number.prototype` মেথড (প্রিমিটিভ নাম্বার ইনস্ট্যান্স)

যখন আপনি একটি প্রিমিটিভ নাম্বারের উপর মেথড কল করেন (যেমন `(123).toString()`), তখন জাভাস্ক্রিপ্ট সাময়িকভাবে একটি `Number` র‍্যাপার অবজেক্ট তৈরি করে এবং এই মেথডগুলো কল করে।

- `toString(radix)`: নির্দিষ্ট বেসে (যেমন বাইনারি, হেক্স) নাম্বারকে স্ট্রিং-এ রূপান্তর করে।
- `toFixed(digits)`: নির্দিষ্ট দশমিক স্থান পর্যন্ত রাউন্ড করে স্ট্রিং রিটার্ন করে।
- `toPrecision(precision)`: নির্দিষ্ট মোট দৈর্ঘ্য (সিগনিফিক্যান্ট ডিজিট) সহ স্ট্রিং রিটার্ন করে।
- `toExponential(fractionDigits)`: সায়েন্টিফিক নোটেশনে স্ট্রিং রিটার্ন করে।
- `valueOf()`: `Number` অবজেক্ট থেকে প্রিমিটিভ নাম্বার রিটার্ন করে।
- `toLocaleString()`: লোকেল-ভিত্তিক ফরম্যাটিং করে।

৪. `BigInt` (আধুনিক সংযোজন)

`Number` টাইপের সীমাবদ্ধতা (নিরাপদ পূর্ণসংখ্যার সীমা) পূরণের জন্য ES2020-এ `BigInt` আসে। এটি `Number`-এর সাথে মিশ্রিত করা যায় না।

- `BigInt` **ফাংশন:** `BigInt(value)` (কনস্ট্রাক্টর নয়, `new` ছাড়া কল করতে হয়)।
- **স্ট্যাটিক মেথড:** `BigInt.asIntN()`, `BigInt.asUintN()`।
- **prototype মেথড:** `toString()`, `toLocaleString()`, `valueOf()`।

সারসংক্ষেপ (কাজের ভিত্তিতে গ্রুপিং)

গ্রুপ	মূল কাজ	ব্যবহারের ধরন
`Number` কনস্ট্রাক্টর	টাইপ কনভার্সন, ভ্যালিডেশন, ধ্রুবক সরবরাহ	`Number.isNaN()`, `Number.MAX_VALUE`
`Math` অবজেক্ট	জটিল গাণিতিক গণনা ও ধ্রুবক	`Math.random()`, `Math.PI`
`Number.prototype`	প্রিমিটিভ নাম্বারকে ফরম্যাট বা কনভার্ট করা	`(123.456).toFixed(2)`
`BigInt`	`Number`-এর সীমার বাইরের বড় পূর্ণসংখ্যা	`BigInt(9007199254740991n)`

একটি গুরুত্বপূর্ণ বিষয়: জাভাস্ক্রিপ্টে `parseInt()` এবং `parseFloat()` গ্লোবাল ফাংশন হিসেবেও আছে, কিন্তু ES6 থেকে `Number.parseInt()` এবং `Number.parseFloat()` ব্যবহার করা প্রস্তাবিত, কারণ এটি কোডকে আরও Modular করে।

JavaScript String Buildin object & Function

১. `String` কনস্ট্রাক্টর ও স্ট্যাটিক মেথড

এগুলো সরাসরি `String` অবজেক্টের অধীনে থাকে এবং `String.` দিয়ে শুরু করে ব্যবহার করতে হয়। মূলত ভ্যালু কনভার্সন বা বিশেষ স্ট্রিং তৈরিতে কাজে লাগে।

- `String()`: যেকোনো ভ্যালুকে স্ট্রিং-এ রূপান্তর করে (ফাংশন হিসেবে কল করলে)।
- `String.fromCharCode(...codeUnits)`: ইউনিকোড কোড ইউনিট থেকে স্ট্রিং তৈরি করে।
- `String.fromCodePoint(...codePoints)`: ইউনিকোড কোড পয়েন্ট থেকে স্ট্রিং তৈরি করে।
- `String.raw(template, ...substitutions)`: টেমপ্লেট লিটারালের কাঁচা (raw) স্ট্রিং রিটার্ন করে।

২. `String.prototype` এর প্রপার্টি (Instance Properties)

স্ট্রিং ইনস্ট্যান্সের সাথে সরাসরি সংযুক্ত একমাত্র প্রপার্টি।

- `length`: স্ট্রিং-এর অক্ষর সংখ্যা রিটার্ন করে (শুধুমাত্র পড়া যায়)।
- `constructor`: স্ট্রিং তৈরি করার কনস্ট্রাক্টর ফাংশন (সাধারণত `String`)।

৩. `String.prototype` এর মেথড (Instance Methods)

এগুলো সবচেয়ে বেশি ব্যবহৃত হয়। কোনো স্ট্রিং ভেরিয়েবলের উপর সরাসরি কল করা যায়। এগুলোকে আরও কয়েকটি উপ-গ্রুপে ভাগ করা যায়:

ক. অক্ষর খোঁজা ও পরীক্ষা করা (Character & Search):

- `charAt(index)`: নির্দিষ্ট ইনডেক্সের অক্ষর রিটার্ন করে।
- `charCodeAt(index)`: নির্দিষ্ট ইনডেক্সের অক্ষরের UTF-16 কোড রিটার্ন করে।
- `codePointAt(pos)`: নির্দিষ্ট পজিশনের ইউনিকোড কোড পয়েন্ট রিটার্ন করে।
- `at(index)`: ইনডেক্সের অক্ষর রিটার্ন করে, ঋণাত্মক ইনডেক্স সাপোর্ট করে।
- `includes(searchString, position)`: স্ট্রিং-এ নির্দিষ্ট সাব-স্ট্রিং আছে কিনা চেক করে (বুলিয়ান)।
- `startsWith(searchString, position)`: নির্দিষ্ট সাব-স্ট্রিং দিয়ে শুরু হয়েছে কিনা চেক করে।
- `endsWith(searchString, endPosition)`: নির্দিষ্ট সাব-স্ট্রিং দিয়ে শেষ হয়েছে কিনা চেক করে।
- `indexOf(searchString, position)`: সাব-স্ট্রিং প্রথম কোথায় পাওয়া যায় তার ইনডেক্স রিটার্ন করে (না পেলে -1)।
- `lastIndexOf(searchString, position)`: সাব-স্ট্রিং শেষ কোথায় পাওয়া যায় তার ইনডেক্স রিটার্ন করে।

খ. স্ট্রিং কাটা, জোড়া লাগানো ও পরিবর্তন (Extraction & Manipulation):

- `slice(start, end)`: স্ট্রিং-এর একটি অংশ কেটে নতুন স্ট্রিং রিটার্ন করে (ঋণাত্মক ইনডেক্স সাপোর্ট করে)।
- `substring(start, end)`: স্ট্রিং-এর একটি অংশ কেটে নতুন স্ট্রিং রিটার্ন করে (ঋণাত্মক ইনডেক্স সাপোর্ট করে না)।
- `substr(start, length)`: ডেপ্রিকেটেড। নির্দিষ্ট দৈর্ঘ্য অনুযায়ী অংশ কেটে নেয়।
- `concat(...args)`: এক বা একাধিক স্ট্রিং জোড়া লাগায়।
- `split(separator, limit)`: নির্দিষ্ট সেপারেটর দিয়ে স্ট্রিং ভেঙে অ্যারে তৈরি করে।
- `replace(searchValue, replaceValue)`: প্রথম ম্যাচ হওয়া অংশ রিপ্লেস করে।
- `replaceAll(searchValue, replaceValue)`: সব ম্যাচ হওয়া অংশ রিপ্লেস করে।
- `repeat(count)`: স্ট্রিং নির্দিষ্ট বার রিপিট করে।
- `padStart(targetLength, padString)`: শুরুতে নির্দিষ্ট ক্যারেক্টার দিয়ে প্যাড করে।
- `padEnd(targetLength, padString)`: শেষে নির্দিষ্ট ক্যারেক্টার দিয়ে প্যাড করে।
- `trim()`: দুই পাশের হোয়াইটস্পেস মুছে দেয়।
- `trimStart() / trimEnd()`: শুধু শুরু বা শুধু শেষের হোয়াইটস্পেস মুছে দেয়।

গ. কেস পরিবর্তন ও তুলনা (Case & Comparison):

- `toLowerCase()`: ছোট হাতের অক্ষরে রূপান্তর করে।
- `toUpperCase()`: বড় হাতের অক্ষরে রূপান্তর করে।
- `toLocaleLowerCase() / toLocaleUpperCase()`: লোকেল অনুযায়ী কেস পরিবর্তন করে।
- `localeCompare(compareString)`: লোকেল অনুযায়ী দুটি স্ট্রিং তুলনা করে।

ঘ. রেগুলার এক্সপ্রেশন (Regex):

- `match(regexp)`: রেগুলার এক্সপ্রেশনের সাথে ম্যাচ খুঁজে বের করে।
- `matchAll(regexp)`: সব ম্যাচের একটি ইটারেটর রিটার্ন করে।
- `search(regexp)`: ম্যাচের ইনডেক্স খুঁজে বের করে।

ঙ. ইউনিকোড ও অন্যান্য (Unicode & Utility):

- `normalize([form])`: ইউনিকোড নরমালাইজেশন ফর্ম রিটার্ন করে।
- `isWellFormed()`: স্ট্রিং-এ কোনো লোন সারোগেট আছে কিনা চেক করে।
- `toWellFormed()`: লোন সারোগেটকে U+FFFD দিয়ে রিপ্লেস করে।
- `toString() / valueOf()`: স্ট্রিং অবজেক্ট থেকে প্রিমিটিভ ভ্যালু রিটার্ন করে।
- `[Symbol.iterator]()`: স্ট্রিং-কে ইটারেট করার সুবিধা দেয়।

৪. `String.prototype` এর HTML র‍্যাপার মেথড (HTML Wrapper Methods)

এগুলো ডেপ্রিকেটেড এবং স্ট্যান্ডার্ড নয়। এগুলো স্ট্রিং-কে HTML ট্যাগে মুড়িয়ে দেয়। আধুনিক ওয়েব ডেভেলপমেন্টে এগুলো ব্যবহার করা উচিত নয়; এর পরিবর্তে DOM মেথড বা টেমপ্লেট লিটারাল ব্যবহার করা হয়।

- `anchor(name)`, `big()`, `blink()`, `bold()`, `fixed()`, `fontcolor(color)`, `fontsize(size)`, `italics()`, `link(url)`, `small()`, `strike()`, `sub()`, `sup()`।

সারসংক্ষেপ: একজন ডেভেলপারের জন্য `length`, `slice()`, `split()`, `includes()`, `indexOf()`, `replace()`, `trim()`, `toUpperCase()`, `toLowerCase()` মেথডগুলো সবচেয়ে বেশি প্রয়োজন হয়।

JavaScript Symbol Buildin object & Function

১. Symbol কনস্ট্রাক্টর ও গ্লোবাল ফাংশন

Symbol তৈরি করার মূল উপায় হলো `Symbol()` ফাংশনটি কল করা। এটি একটি অনন্য (unique) Symbol রিটার্ন করে।

- `Symbol()`: একটি নতুন, অনন্য Symbol তৈরি করে। ঐচ্ছিকভাবে একটি স্ট্রিং ডেসক্রিপশন নিতে পারে (যা শুধু ডিবাগিং-এর জন্য, ইউনিকনেস নষ্ট করে না)। `new Symbol()` ব্যবহার করা যায় না।
- `Symbol.for(key)`: গ্লোবাল Symbol রেজিস্ট্রি থেকে একটি Symbol খোঁজে বা তৈরি করে। একই `key` দিয়ে কল করলে সবসময় একই Symbol রিটার্ন করবে (শেয়ার্ড Symbol)।
- `Symbol.keyFor(sym)`: একটি শেয়ার্ড Symbol-এর জন্য গ্লোবাল রেজিস্ট্রির `key` স্ট্রিং রিটার্ন করে। শুধুমাত্র `Symbol.for()` দিয়ে তৈরি করা Symbol-এর জন্যই কাজ করে।

২. সুপরিচিত Symbol (Well-Known Symbols) — স্ট্যাটিক প্রপার্টি

`Symbol` কনস্ট্রাক্টরের সব স্ট্যাটিক প্রপার্টি নিজেরাই এক-একটি Symbol। এগুলোকে সুপরিচিত Symbol (Well-Known Symbols) বলা হয়। এগুলোর মূল কাজ হলো জাভাস্ক্রিপ্টের কিছু বিল্ট-ইন আচরণকে কাস্টমাইজ করা বা "হুক" প্রদান করা, যা সাধারণ স্ট্রিং প্রপার্টি দিয়ে করা যেত না।

- `Symbol.iterator`: `for...of` লুপ এবং স্প্রেড (`...`) অপারেটর কীভাবে কোনো অবজেক্টকে ইটারেট করবে তা নির্ধারণ করে।
- `Symbol.asyncIterator`: `for await...of` লুপের জন্য অ্যাসিনক্রোনাস ইটারেটর নির্ধারণ করে।
- `Symbol.hasInstance`: `instanceof` অপারেটরের আচরণ নির্ধারণ করে।
- `Symbol.toPrimitive`: কোনো অবজেক্টকে আদিম (primitive) মানে রূপান্তর করার সময় কীভাবে আচরণ করবে তা নির্ধারণ করে।
- `Symbol.toStringTag`: `Object.prototype.toString.call(obj)` কল করলে কী ট্যাগ দেখাবে তা নির্ধারণ করে।
- `Symbol.species`: কোনো কনস্ট্রাক্টর ফাংশন যখন ডেরাইভড (derived) অবজেক্ট তৈরি করে, তখন কোন কনস্ট্রাক্টর ব্যবহার করবে তা নির্ধারণ করে।
- `Symbol.match` / `Symbol.replace` / `Symbol.search` / `Symbol.split`: রেগুলার এক্সপ্রেশন-সম্পর্কিত `String` মেথডগুলোর (`match`, `replace`, `search`, `split`) আচরণ নির্ধারণ করে।
- `Symbol.isConcatSpreadable`: কোনো অবজেক্টকে `Array.prototype.concat()`-এ ফ্ল্যাট করা উচিত কিনা তা নির্ধারণ করে।
- `Symbol.unscopables`: `with` স্টেটমেন্টের স্কোপ থেকে কোন প্রপার্টিগুলো বাদ দেওয়া হবে তা নির্ধারণ করে।
- `Symbol.dispose` / `Symbol.asyncDispose`: রিসোর্স ম্যানেজমেন্টের জন্য `using` / `await using` ডিক্লারেশনে ব্যবহৃত হয় (ES2023+)।

৩. Symbol প্রোটোটাইপের প্রপার্টি ও মেথড (Instance Properties & Methods)

এই মেথডগুলো Symbol অবজেক্টের ইনস্ট্যান্সে (`Object(sym)` দিয়ে তৈরি করা) বা প্রিমিটিভের উপর কল করা যায়।

- `Symbol.prototype.description` **(Getter)**: Symbol-এর ডেসক্রিপশন স্ট্রিং রিটার্ন করে (যেমন `"foo"`), বা `undefined`।
- `Symbol.prototype.toString()`: `"Symbol(description)"` ফরম্যাটে একটি স্ট্রিং রিটার্ন করে।
- `Symbol.prototype.valueOf()`: Symbol অবজেক্ট থেকে প্রিমিটিভ Symbol মান রিটার্ন করে।
- `Symbol.prototype[Symbol.toPrimitive]`: এই Symbol অবজেক্টকে আদিম মানে রূপান্তর করার জন্য কল করা হয়।
- `Symbol.prototype[Symbol.toStringTag]`: `"Symbol"` স্ট্রিং মান ধারণ করে।

৪. Symbol-সংক্রান্ত ইউটিলিটি ও অবজেক্ট আচরণ

Symbol সরাসরি একটি মেথড না হলেও, কিছু গ্লোবাল ফাংশন বা অবজেক্ট মেথড Symbol-এর সাথে কাজ করে।

- `Object.getOwnPropertySymbols(obj)`: কোনো অবজেক্টের নিজস্ব সব Symbol প্রপার্টির একটি অ্যারে রিটার্ন করে।

- `Object.getOwnPropertyNames(obj)`: Symbol প্রপার্টি বাদ দিয়ে শুধু স্ট্রিং প্রপার্টির নাম রিটার্ন করে।
- `for...in` **লুপ**: Symbol প্রপার্টিগুলো এনুমারেট করে না (ignore করে)।
- `JSON.stringify(obj)`: Symbol প্রপার্টিগুলো সম্পূর্ণভাবে উপেক্ষা করে।

সারসংক্ষেপ

গ্রুপ	মূল কাজ	উদাহরণ
কনস্ট্রাক্টর ও গ্লোবাল ফাংশন	নতুন Symbol তৈরি এবং শেয়ার করা	`Symbol()`, `Symbol.for()`, `Symbol.keyFor()`
সুপরিচিত Symbol (স্ট্যাটিক প্রপার্টি)	বিল্ট-ইন ভাষার আচরণ কাস্টমাইজ করা	`Symbol.iterator`, `Symbol.toPrimitive`, `Symbol.hasInstance`
প্রোটোটাইপ প্রপার্টি ও মেথড	Symbol ইনস্ট্যান্স থেকে মান বের করা	`sym.description`, `sym.toString()`, `sym.valueOf()`
ইউটিলিটি ও অবজেক্ট আচরণ	Symbol প্রপার্টি খোঁজা বা উপেক্ষা করা	`Object.getOwnPropertySymbols()`, `for...in`

JavaScript Array Buildin object & Function

জাভাস্ক্রিপ্টের Arrays (অ্যারে) মূলত একটি বিল্ট-ইন গ্লোবাল অবজেক্ট, যা একটি কনস্ট্রাক্টর হিসেবে কাজ করে এবং এর একটি প্রোটোটাইপ রয়েছে যেখানে অসংখ্য মেথড ও প্রপার্টি সংজ্ঞায়িত করা আছে। কাজের ভিত্তিতে এগুলোকে মূলত ৪টি প্রধান গ্রুপে ভাগ করা যায়।

১. স্ট্যাটিক মেথড (Static Methods)

এগুলো সরাসরি `Array` কনস্ট্রাক্টরের উপর কল করা হয় (`Array.` দিয়ে শুরু)। এদের প্রধান কাজ হলো নতুন অ্যারে তৈরি করা বা কোনো কিছু অ্যারে কিনা তা যাচাই করা।

- `Array.isArray(value)`: একটি মান অ্যারে কিনা তা যাচাই করে `true` বা `false` রিটার্ন করে।
- `Array.from(arrayLike, mapFn)`: কোনো ইটারেবল বা অ্যারে-সদৃশ অবজেক্ট (যেমন `NodeList` বা স্ট্রিং) থেকে নতুন অ্যারে তৈরি করে।
- `Array.of(...items)`: পাস করা আর্গুমেন্টগুলো দিয়ে একটি নতুন অ্যারে তৈরি করে। এটি `new Array(5)` এর সমস্যা এড়াতে সাহায্য করে (যা ৫টি খালি স্লট তৈরি করে)।

২. মিউটেটিং মেথড (Mutating Methods)

এই মেথডগুলো মূল অ্যারে পরিবর্তন করে (in-place modification) এবং সাধারণত পরিবর্তিত অ্যারে বা তার সাথে সম্পর্কিত কোনো মান রিটার্ন করে।

- `push(...elements)`: অ্যারের শেষে এক বা একাধিক এলিমেন্ট যোগ করে এবং নতুন `length` রিটার্ন করে।
- `pop()`: অ্যারের শেষ এলিমেন্ট মুছে ফেলে এবং সেই মুছে ফেলা এলিমেন্টটি রিটার্ন করে।

- `shift()`: অ্যারের প্রথম এলিমেন্ট মুছে ফেলে এবং সেটি রিটার্ন করে।
- `unshift(...elements)`: অ্যারের শুরুতে এক বা একাধিক এলিমেন্ট যোগ করে এবং নতুন `length` রিটার্ন করে।
- `splice(start, deleteCount, ...items)`: অ্যারের যেকোনো অবস্থান থেকে এলিমেন্ট মুছে ফেলে, যোগ করে বা প্রতিস্থাপন করে। এটি মুছে ফেলা এলিমেন্টগুলোর একটি অ্যারে রিটার্ন করে।
- `reverse()`: অ্যারের এলিমেন্টগুলোর ক্রম উল্টে দেয় এবং অ্যারের রেফারেন্স রিটার্ন করে।
- `sort(compareFn)`: অ্যারের এলিমেন্টগুলোকে সাজায় এবং অ্যারের রেফারেন্স রিটার্ন করে।
- `fill(value, start, end)`: একটি নির্দিষ্ট মান দিয়ে অ্যারের অংশবিশেষ পূরণ করে।
- `copyWithin(target, start, end)`: অ্যারের ভেতরে নির্দিষ্ট অংশের এলিমেন্টগুলো অন্য জায়গায় কপি করে।

৩. অ্যাক্সেসর বা নন-মিউটেটিং মেথড (Accessor / Non-Mutating Methods)

এই মেথডগুলো মূল অ্যারে পরিবর্তন করে না, বরং নতুন অ্যারে বা স্ট্রিং রিটার্ন করে।

- `slice(start, end)`: অ্যারের একটি অংশ কেটে নতুন অ্যারে তৈরি করে।
- `concat(...values)`: এক বা একাধিক অ্যারে বা মান যোগ করে একটি নতুন অ্যারে তৈরি করে।
- `join(separator)`: অ্যারের সব এলিমেন্টকে একটি স্ট্রিং-এ রূপান্তর করে, মাঝে সেপারেটর বসায়।
- `indexOf(searchElement, fromIndex)`: নির্দিষ্ট এলিমেন্ট প্রথম কোথায় আছে তার ইনডেক্স রিটার্ন করে, না থাকলে `-1`।
- `lastIndexOf(searchElement, fromIndex)`: শেষ থেকে খুঁজে এলিমেন্টের ইনডেক্স রিটার্ন করে।
- `includes(searchElement, fromIndex)`: অ্যারেতে নির্দিষ্ট এলিমেন্ট আছে কিনা `true` বা `false` রিটার্ন করে।
- `flat(depth)`: নেস্টেড অ্যারে (অ্যারের ভেতর অ্যারে) ফ্ল্যাট করে একটি নতুন অ্যারে তৈরি করে।
- `toString()`: অ্যারের এলিমেন্টগুলো কমা দিয়ে জোড়া লাগিয়ে একটি স্ট্রিং রিটার্ন করে।
- `toReversed()`, `toSorted()`, `toSpliced()`, `with()`: এগুলো `reverse`, `sort`, `splice`-এর নন-মিউটেটিং ভার্সন, যা মূল অ্যারে অপরিবর্তিত রেখে নতুন অ্যারে রিটার্ন করে।

৪. ইটারেশন বা কলব্যাক মেথড (Iteration Methods)

এই মেথডগুলো অ্যারের প্রতিটি এলিমেন্টের উপর একটি ফাংশন (callback) চালায়। এগুলোও মূল অ্যারে পরিবর্তন করে না (যদিও callback ফাংশন নিজে পরিবর্তন করতে পারে)।

- `forEach(callbackFn)`: প্রতিটি এলিমেন্টের জন্য একটি ফাংশন চালায়। এটি কিছুই রিটার্ন করে না।
- `map(callbackFn)`: প্রতিটি এলিমেন্টের উপর ফাংশন চালিয়ে ফলাফলের একটি নতুন অ্যারে রিটার্ন করে।
- `filter(callbackFn)`: যে এলিমেন্টগুলো শর্ত পূরণ করে, শুধু সেগুলো দিয়ে নতুন অ্যারে তৈরি করে।
- `reduce(callbackFn, initialValue)`: অ্যারের সব এলিমেন্টকে একটি সিঙ্গেল ভ্যালুতে (যেমন যোগফল) রূপান্তর করে।
- `reduceRight(callbackFn, initialValue)`: ডান থেকে বামে reduce করে।
- `some(callbackFn)`: অ্যারের অন্তত একটি এলিমেন্ট শর্ত পূরণ করলে `true` রিটার্ন করে।
- `every(callbackFn)`: অ্যারের সব এলিমেন্ট শর্ত পূরণ করলে `true` রিটার্ন করে।
- `find(callbackFn)`: শর্ত পূরণ করা প্রথম এলিমেন্টটির মান রিটার্ন করে।
- `findIndex(callbackFn)`: শর্ত পূরণ করা প্রথম এলিমেন্টটির ইনডেক্স রিটার্ন করে।
- `entries()`, `keys()`, `values()`: অ্যারের ইনডেক্স/মানের জন্য ইটারেটর রিটার্ন করে।

প্রপার্টি (Properties)

- `length`: একটি অ্যারের ইনস্ট্যান্স প্রপার্টি, যা অ্যারের এলিমেন্টের সংখ্যা নির্দেশ করে। এটি একটি ডাইনামিক প্রপার্টি এবং এর মান পরিবর্তন করে অ্যারে ছোট বা বড় করা যায়।
- `Array.prototype`: `Array` কনস্ট্রাক্টরের প্রোটোটাইপ, যেখানে সব ইনস্ট্যান্স মেথড ডিফাইন করা থাকে

JavaScript Date Buildin object & Function

জাভাস্ক্রিপ্টে **Date** হলো একটি বিল্ট-ইন গ্লোবাল অবজেক্ট (ক্লাস), যা তারিখ ও সময় নিয়ে কাজ করার জন্য ব্যবহৃত হয়। এটি ১৯৭০ সালের ১ জানুয়ারি UTC থেকে অতিবাহিত মিলিসেকেন্ড (Unix Epoch) সংরক্ষণ করে।

অন্যান্য ডেটা টাইপের মতো Date-এর অনেক বিল্ট-ইন মেথড রয়েছে, কিন্তু কোনো প্রপার্টি নেই। কাজের ভিত্তিতে এগুলোকে মূলত **৫টি প্রধান গ্রুপে** ভাগ করা যায়।

১. স্ট্যাটিক মেথড (Static Methods)

এগুলো সরাসরি `Date` কনস্ট্রাক্টরের উপর কল করা হয় (`Date.` দিয়ে শুরু), কোনো ইনস্ট্যান্স তৈরি না করেই।

- `Date.now()`: বর্তমান সময়ের Unix টাইমস্ট্যাম্প (মিলিসেকেন্ড) রিটার্ন করে। পারফরম্যান্স মাপার জন্য `new Date().getTime()`-এর চেয়ে দ্রুত।
- `Date.parse(dateString)`: একটি তারিখের স্ট্রিং পার্স করে সেই সময়ের মিলিসেকেন্ড রিটার্ন করে।

- `Date.UTC(year, month, ...)`: UTC অনুযায়ী নির্দিষ্ট তারিখের মিলিসেকেন্ড রিটার্ন করে।

২. গেটার মেথড (Getter Methods / "get" Methods)

এগুলো Date ইনস্ট্যান্সের তারিখ ও সময়ের বিভিন্ন অংশ আলাদাভাবে পড়তে ব্যবহৃত হয়। স্থানীয় সময় (Local Time) অনুযায়ী মান দেয়।

- `getFullYear()`: ৪ ডিজিটের বছর (যেমন ২০২৫)।
- `getMonth()`: মাস (০ = জানুয়ারি, ১১ = ডিসেম্বর)।
- `getDate()`: মাসের দিন (১ – ৩১)।
- `getDay()`: সপ্তাহের দিন (০ = রবিবার, ৬ = শনিবার)। এর কোনো setter নেই, কারণ এটি স্বয়ংক্রিয়ভাবে সেট হয়।
- `getHours()`: ঘন্টা (০ – ২৩)।
- `getMinutes()`: মিনিট (০ – ৫৯)।
- `getSeconds()`: সেকেন্ড (০ – ৫৯)।
- `getMilliseconds()`: মিলিসেকেন্ড (০ – ৯৯৯)।
- `getTime()`: Unix Epoch থেকে মিলিসেকেন্ড রিটার্ন করে। তারিখ তুলনার জন্য দরকারি।
- `getTimezoneOffset()`: স্থানীয় সময় ও UTC-এর মধ্যে মিনিটের পার্থক্য রিটার্ন করে।
- **UTC Getter**: `getUTCFullYear()`, `getUTCMonth()`, `getUTCDate()`, `getUTCDay()`, `getUTCHours()`, `getUTCMinutes()`, `getUTCSeconds()`, `getUTCMilliseconds()` — এগুলো UTC অনুযায়ী একই মান দেয়।

৩. সেটার মেথড (Setter Methods / "set" Methods)

এগুলো Date ইনস্ট্যান্সের তারিখ ও সময়ের অংশ পরিবর্তন করতে ব্যবহৃত হয়। মান আউট-অফ-রেঞ্জ হলে, জাভাস্ক্রিপ্ট স্বয়ংক্রিয়ভাবে তারিখ এগিয়ে নেয় বা পিছিয়ে নেয়।

- `setFullYear(year, month, date)`: বছর সেট করে।
- `setMonth(month, date)`: মাস সেট করে।
- `setDate(date)`: মাসের দিন সেট করে।
- `setHours(hour, min, sec, ms)`: ঘন্টা সেট করে।
- `setMinutes(min, sec, ms)`: মিনিট সেট করে।
- `setSeconds(sec, ms)`: সেকেন্ড সেট করে।
- `setMilliseconds(ms)`: মিলিসেকেন্ড সেট করে।
- `setTime(timeValue)`: Unix Epoch থেকে মিলিসেকেন্ড দিয়ে সম্পূর্ণ সময় সেট করে।
- **UTC Setter**: `setUTCFullYear()`, `setUTCMonth()`, `setUTCDate()`, `setUTCHours()`, `setUTCMinutes()`, `setUTCSeconds()`, `setUTCMilliseconds()` — UTC অনুযায়ী সেট করে।

৪. ফরম্যাটিং বা "to" মেথড (Formatting / "to" Methods)

এগুলো Date ইনস্ট্যান্স থেকে ভিন্ন ভিন্ন ফরম্যাটে স্ট্রিং রিটার্ন করে।

- `toString()`: ডিফল্ট, স্থানীয় সময়ের রিডেবল স্ট্রিং (যেমন `Thu Jan 01 1970 00:00:00 GMT+0000`)।
- `toISOString()`: ISO 8601 ফরম্যাটে স্ট্রিং (যেমন `1970-01-01T00:00:00.000Z`)।
- `toJSON()`: `toISOString()`-এর মতোই, `JSON.stringify()`-এর জন্য ব্যবহৃত হয়।
- `toUTCString()`: UTC সময়ের স্ট্রিং (RFC 7231 ফরম্যাট)।
- `toDateString()` / `toTimeString()`: শুধু তারিখ বা শুধু সময়ের অংশ রিটার্ন করে।
- `toLocaleString()` / `toLocaleDateString()` / `toLocaleTimeString()`: ব্যবহারকারীর লোকেল অনুযায়ী ফরম্যাট করে। `Intl.DateTimeFormat`-এর অপশন সাপোর্ট করে।

৫. কনস্ট্রাক্টর (Constructor)

`Date` ফাংশনটি `new` অপারেটরের সাথে ব্যবহার করে ইনস্ট্যান্স তৈরি করা হয়। `new` ছাড়া কল করলে বর্তমান সময়ের স্ট্রিং রিটার্ন করে।

- `new Date()`: বর্তমান তারিখ ও সময়।
- `new Date(milliseconds)`: Epoch থেকে মিলিসেকেন্ড।
- `new Date(dateString)`: তারিখের স্ট্রিং (ব্রাউজার ভেদে পার্সিং ভিন্ন হতে পারে)।
- `new Date(year, monthIndex, day, hours, minutes, seconds, ms)`: আলাদা আলাদা সংখ্যা দিয়ে তারিখ তৈরি। মাস ০ থেকে শুরু।

JavaScript DOM Manipulation

1. Element Selection

Method	Description
`document.getElementById()`	ID দিয়ে একটি element select করে
`document.getElementsByClassName()`	Class দিয়ে HTMLCollection return করে
`document.getElementsByTagName()`	Tag name দিয়ে HTMLCollection return করে
`document.getElementsByName()`	Name attribute দিয়ে NodeList return করে
`document.querySelector()`	CSS selector দিয়ে প্রথম element select করে
`document.querySelectorAll()`	CSS selector দিয়ে সব element NodeList আকারে return করে
`document.forms`	সব form element এর collection
`document.images`	সব image element এর collection
`document.links`	সব link element এর collection
`document.scripts`	সব script element এর collection
`element.closest()`	নিকটতম ancestor খুঁজে বের করে
`element.matches()`	Selector এর সাথে match করে কিনা check করে

2. Element Creation & Insertion

Method	Description
`document.createElement()`	নতুন element তৈরি করে

`document.createTextNode()`	নতুন text node তৈরি করে
`document.createDocumentFragment()`	DocumentFragment তৈরি করে
`document.createComment()`	Comment node তৈরি করে
`document.cloneNode()` / `element.cloneNode()`	Element copy করে
`parent.appendChild()`	শেষ child হিসেবে যুক্ত করে
`parent.append()`	একাধিক node/text যুক্ত করে
`parent.prepend()`	শুরুতে যুক্ত করে
`parent.insertBefore()`	নির্দিষ্ট child এর আগে যুক্ত করে
`element.insertAdjacentElement()`	নির্দিষ্ট position এ element যুক্ত করে
`element.insertAdjacentHTML()`	HTML string insert করে
`element.insertAdjacentText()`	Text insert করে
`element.after()`	Element এর পরে যুক্ত করে
`element.before()`	Element এর আগে যুক্ত করে
`element.replaceWith()`	অন্য element দিয়ে replace করে
`parent.replaceChild()`	নির্দিষ্ট child replace করে

3. Element Removal

Method	Description
`parent.removeChild()`	নির্দিষ্ট child remove করে
`element.remove()`	নিজেই নিজেকে remove করে
`element.replaceChildren()`	সব children replace/remove করে
`element.innerHTML = ''`	ভেতরের সব content মুছে ফেলে

4. Content Manipulation

Property/Method	Description
`element.innerHTML`	HTML content get/set করে
`element.outerHTML`	Element সহ HTML get/set করে
`element.innerText`	Visible text get/set করে
`element.textContent`	সব text (hidden সহ) get/set করে
`element.insertAdjacentHTML()`	নির্দিষ্ট position এ HTML insert করে

5. Attribute Manipulation

Method	Description
`element.getAttribute()`	Attribute এর value পায়
`element.setAttribute()`	Attribute set করে
`element.removeAttribute()`	Attribute remove করে
`element.hasAttribute()`	Attribute আছে কিনা check করে
`element.hasAttributes()`	কোনো attribute আছে কিনা check করে
`element.getAttributeNames()`	সব attribute এর নাম array আকারে দেয়
`element.attributes`	NamedNodeMap return করে
`element.dataset`	`data-*` attribute access করে
`element.toggleAttribute()`	Attribute toggle করে

6. Class Manipulation (ক্লাস)

Property/Method	Description
`element.className`	Class string get/set করে
`element.classList`	DOMTokenList return করে
`classList.add()`	Class যোগ করে
`classList.remove()`	Class মুছে ফেলে
`classList.toggle()`	Class add/remove toggle করে
`classList.contains()`	Class আছে কিনা check করে
`classList.replace()`	একটি class অন্যটি দিয়ে replace করে
`classList.item()`	নির্দিষ্ট index এর class return করে

7. Style Manipulation

Property/Method	Description
`element.style.property`	Inline style set/get করে
`element.style.cssText`	সব inline style string আকারে
`element.style.setProperty()`	Custom property set করে
`element.style.getPropertyValue()`	Property value পায়
`element.style.removeProperty()`	Property remove করে
`getComputedStyle(element)`	Computed style return করে
`element.hidden`	Element hide/show করে

8. Traversal

Property	Description
`element.parentNode`	Parent node
`element.parentElement`	Parent element
`element.childNodes`	সব child node (NodeList)
`element.children`	শুধু element child (HTMLCollection)
`element.firstChild`	প্রথম child node
`element.firstElementChild`	প্রথম element child
`element.lastChild`	শেষ child node
`element.lastElementChild`	শেষ element child
`element.nextSibling`	পরের sibling node
`element.nextElementSibling`	পরের element sibling
`element.previousSibling`	আগের sibling node
`element.previousElementSibling`	আগের element sibling
`element.nodeType`	Node এর type
`element.nodeName`	Node এর নাম
`element.nodeValue`	Node এর value

9. Events

Method/Property	Description
`element.addEventListener()`	Event listener যোগ করে
`element.removeEventListener()`	Event listener সরায়

`element.dispatchEvent()`	Event trigger করে
`new Event()`	Custom event তৈরি করে
`new CustomEvent()`	Custom event (data সহ) তৈরি করে
`event.preventDefault()`	Default behavior বন্ধ করে
`event.stopPropagation()`	Event bubbling বন্ধ করে
`event.stopImmediatePropagation()`	সব listener বন্ধ করে
`event.target`	যেখানে event ঘটেছে
`event.currentTarget`	যেখানে listener attach করা
`event.type`	Event এর type
`event.key`	Keyboard key
`event.code`	Keyboard code
`event.clientX / clientY`	Viewport coordinate
`event.pageX / pageY`	Page coordinate
`event.offsetX / offsetY`	Element relative coordinate
`event.screenX / screenY`	Screen coordinate

Common Events:

`click`, `dblclick`, `mousedown`, `mouseup`, `mouseover`, `mouseout`, `mousemove`, `mouseenter`, `mouseleave`, `contextmenu`, `keydown`, `keyup`, `keypress`, `input`, `change`, `submit`, `reset`, `focus`, `blur`, `load`, `DOMContentLoaded`, `scroll`, `resize`, `wheel`, `touchstart`, `touchend`, `touchmove`, `drag`, `drop`, `dragover`, `dragstart`, `dragend`

10. Form Handling

Property/Method	Description
`form.elements`	সব form element
`form.submit()`	Form submit করে
`form.reset()`	Form reset করে
`form.checkValidity()`	Form valid কিনা check করে
`form.reportValidity()`	Validation message দেখায়
`input.value`	Input এর value
`input.checked`	Checkbox/radio state
`input.selected`	Option selected state
`input.files`	FileList (file input)
`input.setCustomValidity()`	Custom validation message

11. Dimension & Position

Property/Method	Description
`element.offsetWidth / offsetHeight`	Border সহ size
`element.offsetTop / offsetLeft`	Offset parent থেকে দূরত্ব
`element.offsetParent`	Offset parent element
`element.clientWidth / clientHeight`	Padding সহ size
`element.clientTop / clientLeft`	Border width
`element.scrollWidth / scrollHeight`	Scrollable content size
`element.scrollTop / scrollLeft`	Scroll position

`element.scrollIntoView()`	Element এ scroll করে
`element.scrollTo()`	নির্দিষ্ট position এ scroll
`element.scrollBy()`	Relative scroll
`element.getBoundingClientRect()`	Size ও position object
`element.getClientRects()`	সব rect return করে
`window.innerWidth / innerHeight`	Window size
`window.scrollX / scrollY`	Window scroll position
`window.scrollTo()` / `window.scrollBy()`	Window scroll

12. Document Level

Property/Method	Description
`document.documentElement`	`<html>` element
`document.head`	`<head>` element
`document.body`	`<body>` element
`document.title`	Page title
`document.URL`	Current URL
`document.domain`	Domain name
`document.referrer`	Referrer URL
`document.cookie`	Cookies
`document.readyState`	Loading state
`document.activeElement`	Currently focused element
`document.hasFocus()`	Focus আছে কিনা
`document.write()`	Document এ লেখে
`document.open()` / `document.close()`	Document stream
`document.importNode()`	অন্য document থেকে node import
`document.adoptNode()`	Node adopt করে

13. Window Object

Method/Property	Description
`window.alert()`	Alert box
`window.confirm()`	Confirm box
`window.prompt()`	Prompt box
`window.open()`	নতুন window/tab
`window.close()`	Window close করে
`window.focus()` / `window.blur()`	Focus control
`window.print()`	Print dialog
`window.getSelection()`	Selected text
`window.requestAnimationFrame()`	Animation frame
`window.cancelAnimationFrame()`	Animation cancel
`window.setTimeout()` / `clearTimeout()`	Timer
`window.setInterval()` / `clearInterval()`	Interval
`window.localStorage`	Local storage
`window.sessionStorage`	Session storage

`window.history`	History object
`window.location`	Location object
`window.navigator`	Navigator object

14. Storage (স্টোরেজ)

Method	Description
`localStorage.setItem()`	Data save করে
`localStorage.getItem()`	Data পায়
`localStorage.removeItem()`	Data মুছে ফেলে
`localStorage.clear()`	সব data মুছে ফেলে
`localStorage.key()`	নির্দিষ্ট index এর key
`sessionStorage.*`	একই method, session এর জন্য
`indexedDB.open()`	IndexedDB খোলে

15. Miscellaneous

Method/Property	Description
`element.focus()` / `element.blur()`	Focus control
`element.click()`	Programmatic click
`element.contains()`	Child কিনা check করে
`element.isEqualNode()`	দুই node সমান কিনা
`element.isSameNode()`	একই node কিনা
`element.compareDocumentPosition()`	দুই node এর position তুলনা
`element.normalize()`	Text node merge করে
`node.hasChildNodes()`	Child আছে কিনা
`MutationObserver`	DOM পরিবর্তন observe করে
`IntersectionObserver`	Element visibility observe করে
`ResizeObserver`	Element resize observe করে
`NodeIterator` / `TreeWalker`	DOM traverse করে
`Range`	DOM range select করে
`Selection`	Text selection

16. HTMLCollection vs NodeList

Feature	HTMLCollection	NodeList
Live?	সাধারণত Live	সাধারণত Static
Methods	`item()`, `namedItem()`	`forEach()`, `item()`, `entries()`, `keys()`, `values()`
Array method	নেই	`forEach` আছে

17. Common Properties

- `element.id`, `element.className`, `element.tagName`
- `element.children`, `element.childElementCount`
- `element.parentElement`, `element.parentNode`
- `element.nextElementSibling`, `element.previousElementSibling`
- `element.innerHTML`, `element.textContent`, `element.innerText`
- `element.value`, `element.checked`, `element.disabled`

- `element.href`, `element.src`, `element.alt`, `element.title`

18. Timing & Performance

Method	Description
`performance.now()`	High-resolution timestamp
`requestAnimationFrame()`	Frame-based animation
`queueMicrotask()`	Microtask queue
`MutationObserver`	DOM change monitor

---

19. Best Practices

1. `querySelector` / `querySelectorAll` — আধুনিক ও flexible
2. `textContent` ব্যবহার করো `innerHTML` এর বদলে (XSS safe)
3. `addEventListener` ব্যবহার করো inline event এর বদলে
4. `DocumentFragment` ব্যবহার করো bulk insert এ (performance)
5. Event Delegation ব্যবহার করো অনেক child এ
6. `classList` ব্যবহার করো `className` এর বদলে
7. Cache করো DOM reference (বারবার query না করা)
8. `requestAnimationFrame` ব্যবহার করো animation এ

---

20. Quick Cheatsheet

```
// Select
const el = document.querySelector('#id');

// Create
const div = document.createElement('div');

// Insert
parent.appendChild(div);
parent.prepend(div);
el.insertAdjacentElement('beforeend', div);

// Remove
el.remove();
parent.removeChild(div);

// Content
el.textContent = 'Hello';
el.innerHTML = '<b>Hi</b>';

// Attribute
el.setAttribute('data-x', '1');
el.dataset.x = '1';

// Class
el.classList.add('active');
el.classList.toggle('hidden');
```

```
// Style
el.style.color = 'red';

// Event
el.addEventListener('click', (e) => {
  e.preventDefault();
  console.log(e.target);
});

// Traverse
el.parentElement;
el.children;
el.nextElementSibling;

// Dimension
el.getBoundingClientRect();
```

JavaScript BOM Manipulation

BOM হলো Browser Object Model এটি JavaScript এর মাধ্যমে browser এর সাথে interact করার interface। BOM এর কোনো official standard নেই

2. Window Size & Position

Property/Method	Description
`window.innerWidth`	Viewport এর width (scrollbar সহ)
`window.innerHeight`	Viewport এর height
`window.outerWidth`	Browser window এর width
`window.outerHeight`	Browser window এর height
`window.screenX` / `screenLeft`	Screen এ window এর X position
`window.screenY` / `screenTop`	Screen এ window এর Y position
`window.pageXOffset`	Horizontal scroll (alias: scrollX)
`window.pageYOffset`	Vertical scroll (alias: scrollY)
`window.scrollX`	Horizontal scroll position
`window.scrollY`	Vertical scroll position
`window.devicePixelRatio`	Device pixel ratio

3. Window Methods (Opening & Closing)

Method	Description
`window.open()`	নতুন window/tab খোলে

`window.close()`	Window বন্ধ করে
`window.focus()`	Window এ focus আনে
`window.blur()`	Window থেকে focus সরায়
`window.print()`	Print dialog খোলে
`window.stop()`	Page loading বন্ধ করে
`window.moveTo(x, y)`	Window move করে
`window.moveBy(dx, dy)`	Relative move
`window.resizeTo(w, h)`	Window resize করে
`window.resizeBy(dw, dh)`	Relative resize

4. Window Scroll Methods

Method	Description
`window.scroll(x, y)`	নির্দিষ্ট position এ scroll
`window.scrollTo(x, y)`	Absolute scroll
`window.scrollBy(x, y)`	Relative scroll
`window.scrollTo({top, left, behavior})`	Smooth scroll options

5. Dialog Boxes (Dialogs)

Method	Description
`window.alert()`	Alert message দেখায়
`window.confirm()`	OK/Cancel dialog, boolean return
`window.prompt()`	Input dialog, string return

6. Timers

Method	Description
`setTimeout(fn, ms)`	নির্দিষ্ট সময় পরে execute
`clearTimeout(id)`	Timeout cancel
`setInterval(fn, ms)`	নির্দিষ্ট interval এ repeat
`clearInterval(id)`	Interval cancel
`setImmediate(fn)`	(Non-standard, Node.js)
`requestAnimationFrame(fn)`	Frame-based animation
`cancelAnimationFrame(id)`	Animation cancel
`queueMicrotask(fn)`	Microtask queue

7. Location Object (URL)

Property/Method	Description
`location.href`	পুরো URL
`location.protocol`	`http:`, `https:`
`location.host`	Hostname + port
`location.hostname`	Domain name
`location.port`	Port number
`location.pathname`	Path
`location.search`	Query string (`?key=value`)
`location.hash`	Fragment (`#section`)
`location.origin`	Protocol + host
`location.username`	URL এর username
`location.password`	URL এর password
`location.assign(url)`	নতুন page load করে
`location.replace(url)`	History তে না রেখে replace
`location.reload()`	Page reload করে
`location.toString()`	URL string return করে

8. History Object

Property/Method	Description
`history.length`	History entry সংখ্যা
`history.state`	Current state object
`history.back()`	আগের page
`history.forward()`	পরের page
`history.go(n)`	n step আগে/পরে (0 = reload)
`history.pushState(state, title, url)`	নতুন history entry যোগ
`history.replaceState(state, title, url)`	Current entry replace
`history.scrollRestoration`	Scroll restore (`auto`/`manual`)

9. Navigator Object

Property/Method	Description
`navigator.userAgent`	Browser এর user agent string
`navigator.platform`	Platform (Win32, MacIntel)
`navigator.language`	Primary language
`navigator.languages`	সব language array
`navigator.onLine`	Online কিনা (boolean)
`navigator.cookieEnabled`	Cookie enabled কিনা
`navigator.hardwareConcurrency`	CPU core সংখ্যা
`navigator.deviceMemory`	RAM (GB)
`navigator.maxTouchPoints`	Touch points সংখ্যা

`navigator.vendor`	Browser vendor
`navigator.appName`	Browser name (deprecated)
`navigator.appVersion`	Browser version (deprecated)
`navigator.appCodeName`	Code name (deprecated)
`navigator.product`	Engine name
`navigator.geolocation`	Geolocation API
`navigator.clipboard`	Clipboard API
`navigator.mediaDevices`	Media (camera/mic)
`navigator.serviceWorker`	Service Worker
`navigator.permissions`	Permissions API
`navigator.storage`	Storage API
`navigator.sendBeacon()`	Data beacon পাঠায়
`navigator.vibrate()`	Device vibrate
`navigator.share()`	Native share
`navigator.getBattery()`	Battery API
`navigator.credentials`	Credentials API
`navigator.doNotTrack`	DNT setting
`navigator.plugins`	Plugins list
`navigator.mimeTypes`	MIME types

10. Screen Object

Property/Method	Description
`screen.width`	Screen এর width (pixels)
`screen.height`	Screen এর height
`screen.availWidth`	Available width (taskbar বাদে)
`screen.availHeight`	Available height
`screen.colorDepth`	Color depth (bits)
`screen.pixelDepth`	Pixel depth
`screen.orientation`	Orientation object
`screen.orientation.type`	`portrait-primary`, `landscape-primary` ইত্যাদি
`screen.orientation.angle`	Rotation angle

11. Frames & Iframe

Property/Method	Description
`window.frames`	সব frame এর collection
`window.frames.length`	Frame সংখ্যা
`window.frames[0]`	প্রথম frame
`window.parent`	Parent window
`window.top`	Top window
`window.self`	Current window

`iframe.contentWindow`	iframe এর window
`iframe.contentDocument`	iframe এর document
`window.postMessage()`	Cross-origin message
`window.onmessage`	Message receive handler

12. Storage (Storage APIs)

LocalStorage

Method	Description
`localStorage.setItem(k, v)`	Data save
`localStorage.getItem(k)`	Data পাওয়া
`localStorage.removeItem(k)`	Data মুছে ফেলা
`localStorage.clear()`	সব clear
`localStorage.key(i)`	নির্দিষ্ট index এর key
`localStorage.length`	Items সংখ্যা

SessionStorage

Method	Description
`sessionStorage.setItem()`	Session এর জন্য save
`sessionStorage.getItem()`	Session data পাওয়া
`sessionStorage.removeItem()`	মুছে ফেলা
`sessionStorage.clear()`	সব clear

IndexedDB

Method	Description
`indexedDB.open()`	Database খোলে
`indexedDB.deleteDatabase()`	Database মুছে ফেলে
`indexedDB.databases()`	সব DB list

Cookies

Property	Description
`document.cookie`	Cookie get/set

13. Console Object

Method	Description
`console.log()`	সাধারণ log
`console.info()`	Info message
`console.warn()`	Warning
`console.error()`	Error
`console.debug()`	Debug message
`console.table()`	Table আকারে
`console.dir()`	Object properties
`console.group()` / `groupEnd()`	Group
`console.groupCollapsed()`	Collapsed group
`console.time()` / `timeEnd()`	Timing
`console.count()`	Count
`console.trace()`	Stack trace
`console.assert()`	Assertion
`console.clear()`	Clear console

14. Events on Window

Event	Description
`load`	Page load complete
`DOMContentLoaded`	DOM ready
`beforeunload`	Page unload এর আগে
`unload`	Page unload
`resize`	Window resize
`scroll`	Window scroll
`online` / `offline`	Network status
`focus` / `blur`	Window focus
`hashchange`	URL hash change
`popstate`	History change
`storage`	Storage change
`message`	postMessage receive
`error`	Error ঘটলে
`beforeprint` / `afterprint`	Print এর আগে/পরে

15. Geolocation API

Method	Description

`navigator.geolocation.getCurrentPosition()`	একবার position
`navigator.geolocation.watchPosition()`	Continuously watch
`navigator.geolocation.clearWatch()`	Watch বন্ধ

Position object:

- `coords.latitude`, `coords.longitude`
- `coords.accuracy`, `coords.altitude`
- `coords.speed`, `coords.heading`
- `timestamp`

16. Web Storage & Cache APIs

API	Description
`localStorage`	Persistent storage
`sessionStorage`	Session storage
`caches`	CacheStorage API
`caches.open()`	Cache খোলে
`caches.keys()`	Cache list
`caches.delete()`	Cache delete
`navigator.storage.estimate()`	Storage usage
`navigator.storage.persist()`	Persistent storage request

17. Fetch & Network APIs

Method	Description
`fetch(url)`	HTTP request
`XMLHttpRequest`	Old AJAX API
`navigator.sendBeacon()`	Beacon data
`WebSocket`	Real-time connection
`EventSource`	Server-Sent Events
`navigator.onLine`	Online status
`window.addEventListener('online')`	Online event

18. Notification API

Method	Description
`Notification.requestPermission()`	Permission চায়
`new Notification(title, options)`	Notification দেখায়
`Notification.permission`	Current permission

19. Clipboard API

Method	Description
`navigator.clipboard.writeText()`	Text copy
`navigator.clipboard.readText()`	Text paste
`navigator.clipboard.write()`	Rich content
`navigator.clipboard.read()`	Read content

20. Media & Device APIs

API	Description
`navigator.mediaDevices.getUserMedia()`	Camera/Mic access
`navigator.mediaDevices.getDisplayMedia()`	Screen share
`navigator.mediaDevices.enumerateDevices()`	Device list
`navigator.vibrate()`	Vibrate
`navigator.getBattery()`	Battery status
`navigator.share()`	Share API

21. Permissions API

Method	Description
`navigator.permissions.query()`	Permission check
`navigator.permissions.request()`	Permission request

Permission names: `geolocation`, `notifications`, `camera`, `microphone`, `clipboard-read`, `clipboard-write`, `midi`, `push`

22. Performance API

Method	Description
`performance.now()`	High-res timestamp
`performance.timing`	Page timing info
`performance.navigation`	Navigation info
`performance.mark()`	Custom mark
`performance.measure()`	Measurement
`performance.getEntries()`	Performance entries
`performance.memory`	Memory info (Chrome)

23. Crypto API

Property/Method	Description
`crypto.randomUUID()`	Random UUID

`crypto.getRandomValues()`	Random values
`crypto.subtle`	SubtleCrypto API

24. Encoding & URL APIs

Method	Description
`btoa()`	String → Base64
`atob()`	Base64 → String
`encodeURI()`	URI encode
`decodeURI()`	URI decode
`encodeURIComponent()`	Component encode
`decodeURIComponent()`	Component decode
`URL`	URL constructor
`URLSearchParams`	Query string helper

25. Frame & Window Communication

Method	Description
`window.postMessage(message, targetOrigin)`	Cross-origin message
`window.onmessage`	Receive message
`addEventListener('message', handler)`	Message listener

26. Other Window Utilities

Method/Property	Description
`window.getSelection()`	Selected text
`window.getComputedStyle()`	Computed style
`window.matchMedia()`	Media query check
`window.requestIdleCallback()`	Idle callback
`window.cancelIdleCallback()`	Cancel idle
`window.structuredClone()`	Deep clone
`window.isSecureContext`	HTTPS কিনা
`window.customElements`	Custom elements
`window.indexedDB`	IndexedDB access
`window.localStorage`	Storage
`window.sessionStorage`	Storage
`window.crypto`	Crypto API
`window.performance`	Performance API
`window.navigator`	Navigator
`window.screen`	Screen
`window.location`	Location
`window.history`	History

`window.document`	Document
`window.console`	Console
`window.alert/confirm/prompt`	Dialogs
`window.open/close`	Window control

27. BOM vs DOM vs ECMAScript

Feature	BOM	DOM	ECMAScript
Standard	নেই (কিন্তু common)	W3C/WHATWG	ECMA
Top object	`window`	`document`	Global
Purpose	Browser control	HTML control	Language core
Example	`window.open()`	`document.getElementById()`	`Array.map()`

29. Best Practices

1. `window.` prefix সবসময় লিখো না — global scope এ automatically accessible
2. `localStorage` এ sensitive data রাখো না
3. `postMessage` এ target origin সবসময় specify করো (security)
4. `window.open` এর বদলে `<a target="_blank" rel="noopener">` ব্যবহার করো
5. Timers সবসময় clear করো memory leak এড়াতে
6. `navigator.userAgent` parsing এড়িয়ে feature detection করো
7. `beforeunload` ব্যবহার করো না (bad UX)
8. Storage quota সীমিত — `navigator.storage.estimate()` দিয়ে check করো

---

30. Browser Support Notes

- Standard BOM: `window`, `location`, `history`, `navigator`, `screen`, `console`
- Modern APIs: `fetch`, `Notification`, `Geolocation`, `Clipboard`, `Web Crypto`
- Newer APIs: `Permissions`, `Storage`, `Battery`, `Share`
- Experimental: `FileSystem`, `WebUSB`, `WebBluetooth`